CPE 334 Software Engineering

Group Project Deliverable 1 (Sprint 1)

Corporate Expense & Purchase Request Approval System

Theme 5: Workflow and Approval System

Full Lifecycle Software Engineering Specifications & Working Verification Package

Course: CPE 334 Software Engineering (Semester 1/2026)

Submission Date: 14 November 2026

GitHub Repository: https://github.com/ssiriwan/cpe334-approval-system

Release Tag: `v0.1.0-sprint1`

Assessed Commit (main): `fd40bfd4b9b94676100c5c4e7232230752d4c99c`

Team Members

Name	Student ID	GitHub Username
Siriwan Yindeephot (Oum)	67070501075	`@ssiriwan`
Thanabun Tikaew (T)	67070501021	`@thhanabun`

Document Navigation & Key Artifact Index

Part	Section Title	Page	Key Content & Artifacts
Part A	Software Requirements Specification (SRS)	Page 1	FR-01..11, BR-01..07, NFR-01..06, Scope
Part A.2	Feature Inventory	Page 8	F-01..F-11 Definitions, Sprints, Dependencies
Part B	Requirements Traceability Matrix	Page 11	Requirements-to-Feature-to-Test Mapping
Part C	System-Level SDS	Page 15	Monorepo Arch, Prisma Schema, Role Matrix
Part D	Three-Sprint Plan	Page 25	Sprint Breakdown, DoD, RACI Responsibilities
Part E.0	Feature Contract E0 (Auth Foundation)	Page 32	Auth Specification, AC-AUTH-01..05, Tests
Part E.1	Feature Contract E1 (Requester Slice)	Page 36	Requester Specification, AC-01..04b, Tests
Part E.2	Feature Contract E2 (Reviewer Slice)	Page 41	Reviewer Queue & Decisions, AC-05..12, Tests
Part F	Verification & Evidence Package	Page 47	34 Tests, 6 Screenshots, Excerpts, TDD Log

1 Part A: Software Requirements Specification (SRS)

CPE 334 Deliverable 1 — Part A • **Project:** Corporate Expense & Purchase Request Approval System • **Theme:** Theme 5 — Workflow and Approval System • **Authors:** Oum (67070501075) & T (67070501021) • **Status:** Baseline Released (Sprint 1)

1.1 Stakeholder Problem & Chosen Context

Organizations frequently face operational bottlenecks, submission delays, and auditing errors in managing corporate expense reimbursements and purchase requests due to legacy reliance on paper routing and ad-hoc chat channels. This unmanaged process leads to three fundamental failure modes:

1. **No Request Visibility:** Requesters cannot track the status of their submissions, resulting in repeated inquiries, lost documentation, and unmanaged procurement pipelines.
2. **No Audit Trail:** Organizations lack verifiable records of reviewer identities, decision timestamps, and justifications, making compliance tracking and retrospective audits impossible.
3. **Unchecked Conflicts of Interest (Self-Approval Risk):** Reviewers or executives who submit requests can unilaterally approve their own requests (Self-Approval) in the absence of hard automated domain guards.

This product is a dedicated **Workflow and Approval System** engineered to digitize corporate expense and procurement workflows with strict role boundaries: Requester, Reviewer, and Admin (Sprint 3).

1.2 Actors & Responsibilities

Actor	Responsibility	Sprint 1 Scope
Requester	Authenticate into the platform, create expense/purchase requests, inspect personal submissions, view lifecycle status and decision audit history; restricted from calling Reviewer endpoints.	Yes
Reviewer	View assigned pending requests (`PENDING`), execute `APPROVE` or `REJECT` decisions with audit comments; can create personal requests as a requester but **strictly prohibited from self-approving personal requests**.	Yes

Actor	Responsibility	Sprint 1 Scope
Admin	Manage user accounts, assign system roles, and configure department reviewer assignments (data model enum provisioned in Sprint 1; administrative management endpoints scheduled for Sprint 3).	No (Sprint 3)
System	Issue formatted `requestNo` identifiers, assign reviewers deterministically based on category, enforce atomic state transitions, and record append-only audit logs.	Yes

1.3 Product Scope & Explicit Exclusions

1.3.1 In Scope (Whole Product Baseline)

- Request creation and lifecycle tracking (`PENDING`, `APPROVED`, `REJECTED`)
- Reviewer task queue and decision workflow
- Automated reviewer assignment engine
- Immutable append-only approval history and audit trail
- Return for revision and resubmission workflow (Sprint 2)
- Multi-stage hierarchical approval workflow (Sprint 2)
- Document and receipt attachments with secure access control (Sprint 2)
- Search, filter, and aggregate expense reporting (Sprint 3)
- Administrative user account and assignment management (Sprint 3)
- Hardening, automated regression test suites, and containerized deployment (Docker/GCP)

1.3.2 Sprint 1 Slice (Delivered in this Release)

- **F-01:** Authentication & Session (Login/Logout/Current User with HTTP-only cookie)
- **F-02:** Request Creation (Validation, requestNo generation, reviewer assignment)
- **F-03:** My Requests & Data Isolation (View personal requests with isolation guards)
- **F-04:** Review Queue & Decision (Approve/Reject with comment, Self-Approval Guard, State Guard)
- **F-05:** Approval History write (Atomic append-only transaction logging)

1.3.3 Explicit Exclusions (Sprint 1 Deferred Scope)

- Live payment gateway integration and automatic fund disbursement
- External notifications (SMS, Email, or Slack/Teams webhooks)

- Live banking system integrations
- Multi-currency support (exclusively supports Thai Baht - THB)
- Public self-registration or self-service password recovery (uses authenticated seed demo accounts)
- File and receipt attachments in Sprint 1 (deferred to Sprint 2)
- Return for revision workflow in Sprint 1 (deferred to Sprint 2)
- Multi-stage approval hierarchy in Sprint 1 (deferred to Sprint 2)
- Executive analytics dashboard in Sprint 1 (deferred to Sprint 3)
- Real PII or production secrets (all fixtures utilize synthetic mock data)

1.4 Functional Requirements (FR)

ID	Requirement (Observable Behavior)	Sprint	Planned Verification
FR-01	Users can login, logout, and inspect current session user info via HTTP-only cookie; unauthenticated requests to protected endpoints are rejected with 401.	S1	AC-AUTH-01..05 / TEST-AUTH-01..05
FR-02	Authenticated users can create requests with title, amount, category, and description; the system validates inputs, generates unique requestNo, sets PENDING status, and assigns reviewer.	S1	AC-01, AC-02 / TEST-01, TEST-02
FR-03	Requesters can inspect their own submissions list, view status details, and read decision audit history.	S1	AC-03, AC-04 / TEST-03, TEST-04
FR-04	Assigned Reviewers can view their assigned pending queue (PENDING) and execute APPROVE or REJECT decisions with comments; system updates status and records history.	S1	AC-05..AC-12 / TEST-05..TEST-13
FR-05	The system records append-only decision audit history (actor, action, comment, timestamp) within an atomic transaction, providing an audit view.	S1 (write) / S2 (view)	AC-06, AC-11 / TEST-06, TEST-13
FR-06	The system assigns a reviewer to each request; only the assigned reviewer can make a decision; Admins can reconfigure assignments in future sprints.	S1 (assign) / S3 (admin)	AC-05, AC-09 / TEST-05, TEST-11
FR-07	Reviewers can return requests for revision (RETURNED); Requesters can edit fields and resubmit.	S2	Planned S2 Contract

ID	Requirement (Observable Behavior)	Sprint	Planned Verification
FR-08	Multi-stage approval workflow support (e.g. Manager $\rightarrow$ Finance) with separate history records per stage.	S2	Planned S2 Contract
FR-09	Document and receipt attachment upload with secure authorized file retrieval.	S2	Planned S2 Contract
FR-10	Search, filter requests, and export aggregated budget reports by status and category.	S3	Planned S3 Contract
FR-11	Admin user account management, role assignment, and reviewer queue routing configuration.	S3	Planned S3 Contract

1.5 Business Rules (BR)

ID	Rule & Domain Constraint	Failure Behavior	Sprint
BR-01	**No Self-Approval:** Requesters are strictly prohibited from approving or rejecting their own requests, regardless of whether their account possesses REVIEWER or ADMIN privileges.	403 `SELF_APPROVAL_NOT_ALLOWED` (evaluated prior to assignment check)	S1
BR-02	**Valid State Lifecycle:** Requests initialize in `PENDING`; unidirectional transition allowed to `APPROVED` or `REJECTED`; terminal decisions cannot be transitioned again.	400 `INVALID_STATE_TRANSITION`	S1
BR-03	**Data Isolation:** Requesters can only access their own requests; Reviewers can only access requests assigned to them (or requests they created themselves).	403 `ACCESS_DENIED`	S1
BR-04	**Payload Validation:** `title` must be 3–150 characters (trimmed); `amount` must be positive > 0 and $\leq 9,999,999,999.99$ (Decimal(12,2)); `category` must match whitelist; `description` must be 1–1,000 characters.	400 `VALIDATION_ERROR`	S1
BR-05	**Reviewer Assignment:** Every request must have an assigned reviewer; only the designated assigned reviewer has authorization to execute decisions.	403 `ACCESS_DENIED`	S1

ID	Rule & Domain Constraint	Failure Behavior	Sprint
BR-06	**Append-Only History:** Every state-changing action (submit/approve/reject) must append an immutable record to `ApprovalHistory`; modifications/deletions forbidden; status and history update atomically.	No update/delete endpoints; full database transaction rollback on error	S1
BR-07	**Decision Validation:** Decision action must be either `APPROVE` or `REJECT`; `REJECT` requires a non-empty comment between 3–500 characters; `APPROVE` allows optional comment up to 500 characters.	400 `VALIDATION_ERROR`	S1

1.6 Non-Functional Requirements (NFR)

ID	Category	Measurable Target	Verification Method	Sprint Allocation
NFR-01	Security	Passwords hashed with bcrypt cost 12; stateless JWT session stored in HTTP-only Lax Cookie; zero credentials in git.	Security code review, Automated auth test suite (TEST-AUTH-01..05)	Sprint 1
NFR-02	Performance	API p95 response time < 300 ms under 50 concurrent requests; zero N+1 database queries via eager loading.	Benchmark timing script, Prisma query log review	S1 (Design) / S3 (Load)
NFR-03	Persistence	Zero data loss on server restart; transactional consistency; repeatable seed scripts.	Container restart test, Seed replay test	Sprint 1
NFR-04	Error Safety	100% of invalid inputs return standard error JSON with HTTP 4xx and unambiguous machine-readable error code.	Negative API integration test suite (TEST-02, TEST-07..12)	Sprint 1
NFR-05	Testability	Automated integration test coverage covers 100% of Sprint 1 Acceptance Criteria.	Vitest suite (34 tests) + Playwright E2E suite (3 scenarios)	Sprint 1

ID	Category	Measurable Target	Verification Method	Sprint Allocation
NFR-06	Deployability	Single container Docker build; configuration via environment variables; health check endpoint.	Docker container build, `/api/health` HTTP 200 verification	S1 (Design) / Lab 5

1.7 Assumptions, Decisions & Open Questions

ID	Type	Title	Description & Rationale
D-01	Decision	Theme Selection	Theme 5: Workflow and Approval System based on student ID endings (Oum ends in 5, T ends in 1).
D-02	Decision	Architecture Pattern	Single-origin monorepo (Express API + React Vite SPA) sharing unified types and zero-CORS cookie delivery.
D-03	Decision	Security Baseline	HTTP-only, SameSite=Lax JWT authentication cookies; bcrypt cost 12 password hashing; no plaintext credentials stored.
D-04	Decision	Strict No-Self-Approval	Hard-coded domain rule preventing users from reviewing their own submissions, tested via positive and negative suites.
D-05	Decision	Decimal Precision	Currency values stored using `Decimal(12,2)` to prevent floating-point rounding inaccuracies.
D-06	Decision	Category Whitelist	4 whitelisted categories: `EQUIPMENT`, `TRAVEL`, `SOFTWARE`, `SUPPLIES` for structured routing and validation.
D-07	Decision	No File Upload in Sprint 1	Document attachment system deferred to Sprint 2 according to release baseline.
D-08	Assumption	Synthetic Demo Data Only	No live banking, SMS, or payment gateways; all operations run on synthetic test data per academic guidelines.
D-09	Decision	Stateless JWT in Cookie	Logout is handled client-side by clearing cookies; server token blacklist deferred (documented under Known Limitations).
D-10	Decision	Request Number Format	Identifiers follow format `REQ-YYYYMMDD-NNNN` with unique constraint and retry loop to prevent collisions.

ID	Type	Title	Description & Rationale
D-11	Decision	Comment Requirement on Rejection	Mandatory comment on reject to maintain audit accountability; optional on approval.
D-12	Decision	Assigned Queue Only	Reviewers view only their explicitly assigned items rather than all organizational requests, ensuring departmental data segregation.
D-13	Assumption	Demo Credentials	All seeded demonstration test accounts utilize password `Password123!` for local development environments.
D-14	Decision	Atomic Status & History Update	State transitions and audit history insertions are executed atomically within a single `prisma.$transaction`.
Q-01	Open (S3)	Global Admin Visibility	Determine whether global administrators should view all organizational requests across departments (to be resolved in F-10).
Q-02	Open (S2)	Return for Revision Rules	Determine whether resubmitted requests return to the original reviewer or restart the approval workflow.
Q-03	Open (S2)	Attachment Storage Strategy	Establish maximum upload size limits and permitted MIME types for document attachments in Sprint 2.

2 Part A.2: Feature Inventory (F-01 to F-11)

CPE 334 Deliverable 1 — Part B (Feature Inventory) • **Project:** Corporate Expense & Purchase Request Approval System • **Theme:** Theme 5 — Workflow and Approval System • **Authors:** Oum (67070501075) & T (67070501021) • **Status:** Baseline Released (Sprint 1)

2.1 Feature Inventory Table

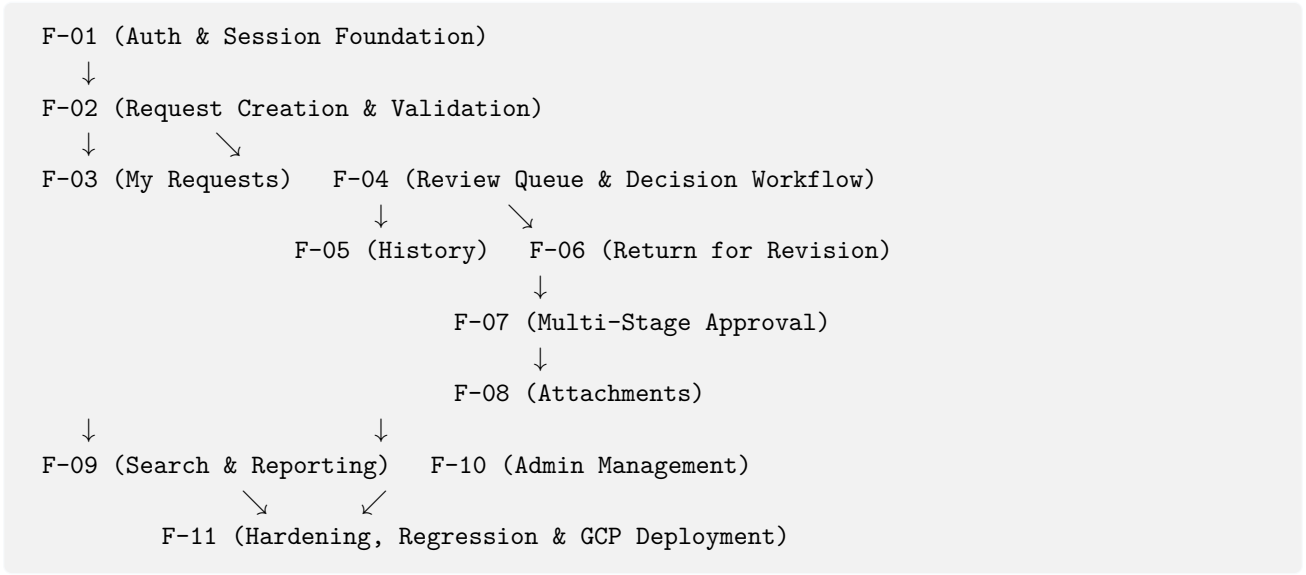
Complete inventory of all product features from conception to final delivery, specifying linkages to Requirements, Dependencies, and Sprint execution plans:

Feat ID	Feature Name	Description	Related Reqs	Depend	Sprint	Planned Verification
F-01	**Auth & Session Management**	User login, logout, and identity inspection via HTTP-only JWT Cookie, preventing unauthorized access to protected endpoints.	FR-01, NFR-01, NFR-04	None	**Sprint 1**	Automated API Integration Tests (TEST-AUTH-01..05)
F-02	**Request Creation & Validation**	Users submit expense/purchase requests with title, amount, category, with automated validation, request number generation, and reviewer assignment.	FR-02, BR-04, BR-05, NFR-04	F-01	**Sprint 1**	API Validation & Persistence Tests (TEST-01, TEST-02), E2E-01
F-03	**My Requests & Data Isolation**	Requesters inspect personal requests and details, enforced by data isolation guards preventing other users from accessing personal records.	FR-03, BR-03, NFR-03	F-01, F-02	**Sprint 1**	API Data Isolation Tests (TEST-03, TEST-04), E2E-01
F-04	**Review Queue & Decision Workflow**	Reviewers inspect assigned queue items, approve or reject with comments, enforcing strict no-self-approval domain rules.	FR-04, FR-06, BR-01, BR-02, BR-05, BR-06, BR-07	F-01, F-02	**Sprint 1**	Domain Rule Tests (TEST-05..TEST-13), E2E-02, E2E-03

Feat ID	Feature Name	Description	Related Reqs	Depend	Sprint	Planned Verification
F-05	**Approval History & Audit Trail**	Records submission, approval, and rejection history in append-only format within atomic transactions, with audit log presentation.	FR-05, BR-06, NFR-03	F-04	**S1 (Write) / S2 (View)**	Database Transaction & Immutability Test (TEST-06, TEST-13)
F-06	**Return for Revision & Resubmission**	Reviewers can return requests for applicant revision, and applicants can update and resubmit modified requests for re-review.	FR-07, BR-02, BR-06	F-04	**Sprint 2**	Sprint 2 Contract & Integration Tests
F-07	**Multi-Stage Approval Workflow**	Multi-stage sequential approval workflow (e.g., Department Manager $\rightarrow$ Finance Director) for high-value requests.	FR-08, BR-02, BR-05, BR-06	F-04, F-06	**Sprint 2**	Sprint 2 Multi-stage Decision Tests
F-08	**Document & Receipt Attachments**	Requesters can upload receipts and quotation documents, enforced by secure role-based access control policies.	FR-09, BR-03, NFR-01	F-02, F-04	**Sprint 2**	Sprint 2 Attachment API & Auth Tests
F-09	**Search, Filter & Expense Reporting**	Search, filter requests, and generate aggregated department expense reports by status, category, and date range.	FR-10, NFR-02	F-03, F-04, F-05	**Sprint 3**	Sprint 3 Reporting & Aggregate Query Tests
F-10	**Admin User & Assignment Management**	System administrators manage user accounts, roles, departments, and category-to-reviewer assignment rules.	FR-11, BR-05	F-01, F-04	**Sprint 3**	Sprint 3 Admin RBAC & Management Tests
F-11	**System Hardening, Regression & Deployment**	Performance tuning, full automated regression test suites, Docker containerization, and GCP cloud deployment.	NFR-01..NFR-06	All (F-01..10)	**Sprint 1 (Base) / Sprint 3**	Continuous Integration, Benchmark Suite, Health Probes

2.2 Feature Dependency Graph

Hierarchical dependency structure between product features showing how each layer builds upon prior functionality:



2.3 Sprint Allocation Summary

Features are mapped into three release increments to guarantee early delivery of the core value proposition:

- **Sprint 1 (Deliverable 1 — Delivered & Released):**
- `F-01` Authentication & Session Management
- `F-02` Request Creation & Field Validation
- `F-03` My Requests & Data Isolation
- `F-04` Review Queue & Decision Workflow
- `F-05` (Write part) Atomic Append-Only Approval History
- **Sprint 2 (Deliverable 2 — Planned):**
- `F-05` (View part) Audit Trail UI View
- `F-06` Return for Revision & Resubmission
- `F-07` Multi-Stage Approval Workflow
- `F-08` Document & Receipt Attachments
- **Sprint 3 (Final Delivery — Planned):**
- `F-09` Search, Filter & Expense Reporting
- `F-10` Admin User & Assignment Management
- `F-11` System Hardening, Regression Suite & GCP Deployment

3 Part B: Requirements Traceability Matrix

CPE 334 Deliverable 1 — Part B (Traceability) • **Project:** Corporate Expense & Purchase Request Approval System • **Theme:** Theme 5 — Workflow and Approval System • **Authors:** Oum (67070501075) & T (67070501021) • **Status:** Baseline Released (Sprint 1)

3.1 Requirements-to-Feature-to-Sprint Traceability Matrix

Complete bidirectional traceability mapping between **Requirements (FR, BR, NFR)** → **Features (F-01..11)** → **Responsible Sprint** → **Acceptance Criteria (AC) & Test IDs**:

3.1.1 Functional Requirements (FR) Traceability

Req ID	Requirement Description	Linked Feature	Sprint	Acceptance Criteria (AC)	Verification Method & Test IDs	Baseline Status
FR-01	User Authentication & Cookie Session	F-01	**Sprint 1**	AC-AUTH-01, AC-AUTH-02, AC-AUTH-03, AC-AUTH-04, AC-AUTH-05	Integration Tests: `TEST-AUTH-01` to `TEST-AUTH-05`	**Verified (Sprint 1, `2bcbf4a`)**
FR-02	Request Creation & Field Validation	F-02	**Sprint 1**	AC-01, AC-02	API & DB Tests: `TEST-01`, `TEST-02`, E2E: `E2E-01`	**Verified (Sprint 1, `2bcbf4a`)**
FR-03	My Requests & Detail Tracking	F-03	**Sprint 1**	AC-03, AC-04	API Isolation Tests: `TEST-03`, `TEST-04`, E2E: `E2E-01`	**Verified (Sprint 1, `2bcbf4a`)**
FR-04	Review Queue & Decision Workflow	F-04	**Sprint 1**	AC-05, AC-06, AC-07, AC-08, AC-09, AC-10, AC-12	Integration Tests: `TEST-05` to `TEST-13`, E2E: `E2E-02`, `E2E-03`	**Verified (Sprint 1, `2bcbf4a`)**
FR-05	Approval History & Audit Trail	F-05	**S1 (Write) / S2 (View)**	AC-06, AC-11	Transaction Test: `TEST-06`, Persistence: `TEST-13`	**Write Verified (S1, `2bcbf4a`); view Sprint 2**
FR-06	Reviewer Assignment	F-04, F-10	**S1 (Assign) / S3 (Admin)**	AC-01, AC-05, AC-09	Assignment Tests: `TEST-01`, `TEST-05`, `TEST-11`	**Assign Verified (S1, `2bcbf4a`); admin Sprint 3**
FR-07	Return for Revision & Resubmission	F-06	**Sprint 2**	Planned in Sprint 2 Contract	Unit & Integration Tests (Sprint 2)	Planned (Sprint 2)

Req ID	Requirement Description	Linked Feature	Sprint	Acceptance Criteria (AC)	Verification Method & Test IDs	Baseline Status
FR-08	Multi-Stage Approval Workflow	F-07	**Sprint 2**	Planned in Sprint 2 Contract	Multi-stage Transition Tests (Sprint 2)	Planned (Sprint 2)
FR-09	Document Attachments	F-08	**Sprint 2**	Planned in Sprint 2 Contract	Multipart Upload & Auth Tests (Sprint 2)	Planned (Sprint 2)
FR-10	Search, Filter & Expense Reporting	F-09	**Sprint 3**	Planned in Sprint 3 Contract	Reporting & Aggregate Query Tests (Sprint 3)	Planned (Sprint 3)
FR-11	Admin User & Role Management	F-10	**Sprint 3**	Planned in Sprint 3 Contract	Admin Access & Role Management Tests (Sprint 3)	Planned (Sprint 3)

3.1.2 Business Rules (BR) Traceability

Rule ID	Business Rule / Domain Guard	Linked Feature	Sprint	Acceptance Criteria (AC)	Verification Method & Test IDs	Baseline Status
BR-01	**No Self-Approval:** Strictly prohibit self-approval	F-04	**Sprint 1**	AC-07	Backend Guard Test: `TEST-08`, E2E: `E2E-03`	**Verified (Sprint 1, `2bcbf4a`)**
BR-02	**Valid State Lifecycle:** `PENDING` $\rightarrow$ `APPROVED` / `REJECTED`	F-04, F-06, F-07	**Sprint 1 / S2**	AC-08 (S1)	State Transition Test: `TEST-09`	**Verified S1 (`2bcbf4a`)**
BR-03	**Data Isolation:** Access limited strictly to authorized requests	F-03, F-04, F-08	**Sprint 1 / S2**	AC-04, AC-09	Forbidden Access Tests: `TEST-04`, `TEST-11`	**Verified S1 (`2bcbf4a`)**
BR-04	**Payload Validation:** Title, Amount > 0, Whitelist Category	F-02	**Sprint 1**	AC-02	Boundary & Negative Tests: `TEST-02`	**Verified (Sprint 1, `2bcbf4a`)**
BR-05	**Reviewer Assignment:** Enforce decisions by assigned reviewer only	F-02, F-04, F-10	**Sprint 1 / S3**	AC-01, AC-05, AC-09	Assignment & Unrelated Reviewer Tests: `TEST-05`, `TEST-11`	**Assign Verified (S1, `2bcbf4a`)**

Rule ID	Business Rule / Domain Guard	Linked Feature	Sprint	Acceptance Criteria (AC)	Verification Method & Test IDs	Baseline Status
BR-06	**Append-Only History:** Immutable transaction logs; no edit/delete	F-04, F-05	**Sprint 1**	AC-06, AC-11	Immutability & Persistence Test: `TEST-06`, `TEST-13`	**Verified (Sprint 1, `2bcbf4a`)**
BR-07	**Decision Validation:** Approve/Reject format + Reason on Reject	F-04	**Sprint 1**	AC-12	Decision Payload Validation Test: `TEST-12`	**Verified (Sprint 1, `2bcbf4a`)**

3.1.3 Non-Functional Requirements (NFR) Traceability

NFR ID	Quality Category	Target & Measure	Linked Feature	Sprint	Verification Method & Evidence
NFR-01	Security	bcrypt cost 12, JWT 8h, httpOnly Lax Cookie, no plaintext passwords	F-01, F-04, F-11	**Sprint 1**	Code audit, `TEST-AUTH-01..05`, Secret scan inspection
NFR-02	Performance	API latency p95 < 300 ms under 50 concurrent requests, no N+1 query	F-01..F-04, F-09, F-11	**S1 / S3**	Local execution timing benchmark scripts
NFR-03	Persistence & Durability	Data and history persist across restarts; repeatable migration & seed	F-02..F-05, F-11	**Sprint 1 / Lab 5**	Database restart verification, clean DB seed execution
NFR-04	Error Safety & Validation	100% of invalid inputs return 4xx with clear machine-readable error codes	F-01..F-04, F-11	**Sprint 1**	Negative integration test suites (`TEST-02`, `TEST-12`, `TEST-AUTH-02`)
NFR-05	Testability & Traceability	100% of Sprint 1 ACs covered by automated tests	F-01..F-04	**Sprint 1**	Traceability matrix cross-check + Vitest suite (34 tests) & Playwright E2E-01..03 (all green locally, PR #16)
NFR-06	Deployability & Config	Containerized deployment, zero hardcoded secrets, health probe	F-11	**S1 (Design) / Lab 5**	Dockerfile inspection, `/api/health` probe test

3.2 Coverage Gaps & Honest Disclosure

In accordance with CPE 334 course governance principles, requirements and features that have not yet undergone full implementation or verification are transparently disclosed:

1. **Requirements Tracking Status (All Requirements Accounted For):**

- Every Requirement (`FR-01..FR-11`, `BR-01..BR-07`, `NFR-01..NFR-06`) is fully inventoried in this document and assigned to an explicit delivery sprint.
- **No untracked requirements exist in the project baseline.**

1. **Sprint 1 Implemented & Verified Scope:**

- `FR-01` through `FR-04`, `FR-05 (Write)`, `FR-06 (Assignment)`, and `BR-01..BR-07` constitute the complete implementation scope of Sprint 1 (via Issues #3, #4, and #5). Baseline status is fully updated to **Verified** backed by test execution evidence and merged PRs.

1. **Later Sprints Scope (Honest Planned Disclosure):**

- `FR-07` (Return for revision), `FR-08` (Multi-stage), and `FR-09` (Attachments) are marked as **Planned for Sprint 2**.
- `FR-10` (Reporting) and `FR-11` (Admin) are marked as **Planned for Sprint 3**.
- Acceptance Criteria and executable test suites for these items are deferred to the detailed Feature Engineering Contracts of Sprints 2 and 3 respectively.

3.3 Approved Change Log

Formal register of specification and scope modifications agreed upon by the engineering team (Approved Changes):

Change ID	Date	Requiremer / Feature	Change Description	Justification & Impact	Approved By	Status
CR-01	2026-11-04	Scope / Theme	Theme 5: Workflow and Approval System	Based on student ID endings (Oum ends in 5, T ends in 1); mutually agreed on Theme 5	Oum & T	Approved
CR-02	2026-11-04	FR-02 / Role Matrix	Allow `REVIEWER` role to submit requests	Crucial for validating `BR-01 No Self-Approval` in realistic production scenarios	Oum & T	Approved
CR-03	2026-11-04	BR-05 / F-02	Category-based Reviewer Assignment	Ensures deterministic reviewer assignment and reliable negative testing for Unrelated Reviewers	Oum & T	Approved
CR-04	2026-11-04	FR-05 / Scope	Bifurcate F-05: Write (S1), View (S2)	Defer redundant UI complexity in Sprint 1 while ensuring 100% atomic transaction logging in DB	Oum & T	Approved

4 Part C: System-Level Software Design Specification (SDS)

Project: CPE 334 - Workflow and Approval System (Theme 5) • **Deliverable:** Deliverable 1 / Sprint 1 • **Owner:** T (67070501021) • **Reviewer:** Oum (67070501075) • **Related issue:** #2 - System-Level SDS and Three-Sprint Plan • **Status:** v1.0 baseline, reviewed before Sprint 1 feature implementation

4.1 Purpose and Scope

This document defines the system-level design for the Corporate Expense and Purchase Request approval system. It describes the architecture, data model, role and permission rules, UI structure, API conventions, authentication lifecycle, configuration, deployment approach, and security expectations.

Sprint 1 implements a working vertical slice:

- authentication and session handling;
- request creation;
- requester-owned request tracking;
- reviewer assignment and review decisions;
- append-only approval history;
- backend enforcement of self-approval, state, validation, and access-control rules.

The design is intentionally a modular monolith. It does not use microservices, message queues, external caches, real payment integrations, or external notification services in Sprint 1.

4.2 Architecture and Technology Stack

```
React 19 + Vite + TypeScript + Tailwind CSS (client)
        |
        | HTTPS / JSON, httpOnly JWT cookie
        v
Node.js + Express + TypeScript (API and application layer)
        |
        | Prisma ORM
        v
PostgreSQL 17 (persistent database)
```

4.2.1 Component responsibilities

Component	Responsibility
Client (`client/`)	Render role-aware screens, collect input, mirror validation feedback, call REST APIs, display loading/empty/error states. UI hiding is not an authorization mechanism.
Server (`server/`)	Authenticate requests, enforce roles and business rules, validate payloads, manage request lifecycle, write history, return standardized API responses.

Component	Responsibility
Prisma	Type-safe database access, migrations, seed support, relationship loading, transaction support.
PostgreSQL	Durable storage for users, approval requests, and approval history.

4.2.2 Architecture decisions

- Business rules live in server controllers/services, never only in the client.
- Client and server run as separate development servers.
- Production builds the client into static assets and runs the server as a container; the client calls the server over HTTP.
- No microservices and no external queue/cache are required for Sprint 1.
- All secrets are supplied through environment variables; no secret is committed or baked into an image.

4.3 Data Model, Constraints, and Persistence

4.3.1 Entities

Entity	Purpose
`User`	Authenticated actor with role `REQUESTER`, `REVIEWER`, or future `ADMIN`.
`ApprovalRequest`	Expense/purchase request owned by a requester and assigned to a reviewer.
`ApprovalHistory`	Append-only audit record for submission, approval, or rejection.

4.3.2 Relationships and ownership

- `ApprovalRequest.requesterId -> User.id`: the user who created the request.
- `ApprovalRequest.assignedReviewerId -> User.id`: the reviewer allowed to decide the request.
- `ApprovalHistory.actorId -> User.id`: the user who performed the action.
- `ApprovalHistory.requestId -> ApprovalRequest.id` with cascade delete from request to history.

4.3.3 Constraints

Area	Constraint
User identity	`User.email` is unique. `passwordHash` stores a bcrypt hash, never plaintext.
Request identity	`ApprovalRequest.requestNo` is unique, formatted as `REQ-YYYYMMDD-NNNN`.
Amount	Stored as `Decimal(12,2)`; service validation requires `amount > 0` and `amount <= 9,999,999,999.99`.
Title	Trimmed length must be 3-150 characters.

Area	Constraint
Category	Must be one of `EQUIPMENT`, `TRAVEL`, `SOFTWARE`, `SUPPLIES`.
Description	Required, non-empty, maximum 1,000 characters.
Status	Starts at `PENDING`; valid transitions are only `PENDING -> APPROVED` or `PENDING -> REJECTED`.
Reviewer assignment	`assignedReviewerId` is nullable at database level for migration safety, but the service must set it when creating a request.
History	Append-only. No update/delete endpoint is exposed in Sprint 1. Status update and history insert must share one transaction.
Decision	`decision` must be `APPROVE` or `REJECT`; `REJECT` requires a 3-500 character comment; `APPROVE` comment is optional and at most 500 characters.

4.3.4 Indexes

- `(requesterId, createdAt)` for "My Requests" ordering.
- `(assignedReviewerId, status)` for the reviewer's pending queue.
- `(status, createdAt)` for status-based views and future reporting.
- `(requestId, createdAt)` for request history retrieval.

4.3.5 Migrations and seed data

- Local development: `npx prisma migrate dev`.
- Released/production environments: `npx prisma migrate deploy`.
- Seed: `prisma/seed.ts` uses idempotent upserts for three synthetic demo accounts.
- Demo requests are created through the UI/API; financial records are not hard-coded in the seed.
- Durability: PostgreSQL volume or Cloud SQL; data and history must survive process restart and redeploy.

4.3.6 Demo accounts

Email	Role	Purpose
`oum.requester@cpe334.local`	`REQUESTER`	Requester flows.
`t.reviewer@cpe334.local`	`REVIEWER`	Reviewer for `EQUIPMENT` and `TRAVEL`.
`manager.hybrid@cpe334.local`	`REVIEWER`	Reviewer for `SOFTWARE` and `SUPPLIES`; self-approval test account.

All demo accounts use a synthetic local password documented in the README. No production credentials are used.

4.4 Roles and Permission Matrix

4.4.1 Principles

- Every protected endpoint checks authentication on the backend.

- Authorization checks role, ownership, assignment, and state in the required order.
- Hiding a button is not authorization; the API must return the correct 401/403/400 error.
- REVIEWER may create requests in Sprint 1 so that BR-01 no-self-approval can be tested with a real reviewer-owned request.
- ADMIN is present in the enum for future work only; it is not seeded and has no Sprint 1 endpoint.

4.4.2 Endpoint permission matrix

Endpoint	Method	Allowed roles	Access rule	Failure behavior
/api/auth/login	POST	Public	Validate email/password, issue JWT cookie.	400 VALIDATION_ERROR; 401 UNAUTHORIZED
/api/auth/logout	POST	Any authenticated user	Clear the token cookie.	401 UNAUTHORIZED
/api/auth/me	GET	Any authenticated user	Return current user without passwordHash.	401 UNAUTHORIZED
/api/requests	POST	REQUESTER, REVIEWER	Create request, generate requestNo, assign reviewer by category.	401 UNAUTHORIZED; 400 VALIDATION_ERROR
/api/requests/my	GET	Any authenticated user	Return only requesterId = currentUser.id, newest first.	401 UNAUTHORIZED
/api/requests/:id	GET	Owner or assigned reviewer	Requester sees own request; reviewer sees only assigned request.	401 UNAUTHORIZED; 403 ACCESS_DENIED; 404 NOT_FOUND
/api/requests/pending	GET	REVIEWER	Return only assignedReviewerId = currentUser.id and status = PENDING.	401 UNAUTHORIZED; 403 FORBIDDEN
/api/requests/:id/review	POST	REVIEWER	Must not be owner; must be assigned reviewer; request must be PENDING; decision must be valid.	401; 403 FORBIDDEN; 403 SELF_APPROVAL_NOT_ALLOWED; 403 ACCESS_DENIED; 400 INVALID_STATE_TRANSITION; 400 VALIDATION_ERROR
/api/health	GET	Public	Health check.	No auth required

4.4.3 Review endpoint check order

The backend must enforce checks in this order:

1. Authentication → 401 UNAUTHORIZED.
2. Role is REVIEWER → 403 FORBIDDEN.
3. Current user is not the request owner → 403 SELF_APPROVAL_NOT_ALLOWED.

4. Current user is the assigned reviewer → `403 ACCESS_DENIED`.
5. Request status is `PENDING` → `400 INVALID_STATE_TRANSITION`.
6. Decision payload is valid → `400 VALIDATION_ERROR`.

Checking self-approval before assignment ensures BR-01 is enforced even when the request is assigned to another reviewer.

4.4.4 Reviewer assignment rule

Sprint 1 uses a deterministic category mapping:

Category	Assigned reviewer
`EQUIPMENT`, `TRAVEL`	`t.reviewer@cpe334.local`
`SOFTWARE`, `SUPPLIES`	`manager.hybrid@cpe334.local`

If the requester is the mapped reviewer, the system assigns the other reviewer instead. Admin reassignment is deferred to Sprint 3.

4.5 UI Structure and Conventions

4.5.1 Screens and routes

Route	Screen	Access	Key states
`/login`	`LoginPage`	Public	default, loading, invalid credentials, server error
`/requests/new`	`NewRequestPage`	Authenticated	Validation, submit, redirect, API error
`/requests/my`	`MyRequestsPage`	Authenticated	loading, empty list, populated list, error
`/requests/:id`	`RequestDetailPage`	Owner or assigned reviewer	loading, not found, forbidden, fields, history
`/reviewer/pending`	`PendingReviewsPage`	`REVIEWER`	loading, empty queue, assigned list, error
`/reviewer/requests/:id`	`ReviewDetailPage`	Assigned reviewer	request detail, approve/reject form, validation, forbidden, conflict
`/403`, `/404`	Error pages	All	static error state

4.5.2 Shared UI behavior

- Client-side validation mirrors server validation, but the server is the source of truth.
- Submit controls are disabled while a request is in flight.

- API errors display a banner or field-level message using the server error code/message.
- Empty, loading, and error states are explicit on every list/detail screen.
- No optimistic status change: the UI updates after a successful API response.
- Role-aware navigation is a usability aid, not a security boundary.
- The UI must not display `passwordHash`, tokens, or other sensitive fields.

4.6 API Approach and Conventions

4.6.1 Response shape

Success:

```
{ "data": { "...": "..." } }
```

Failure:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable description",
    "details": []
  }
}
```

`details` is optional and may contain field-level validation information.

4.6.2 HTTP status codes

Status	Meaning
200	Successful read/update/decision.
201	Request created.
400	Validation failure or invalid state transition.
401	Missing, invalid, or expired authentication.
403	Authenticated but not allowed.
404	Resource not found.
500	Unexpected server error; no stack trace or secret in the response.

4.6.3 Endpoint summary

Endpoint	Method	Purpose
`/api/auth/login`	POST	Login and set the JWT cookie.
`/api/auth/logout`	POST	Clear the JWT cookie.
`/api/auth/me`	GET	Return current user.
`/api/requests`	POST	Create a request.

Endpoint	Method	Purpose
`/api/requests/my`	GET	List current user's requests.
`/api/requests/:id`	GET	Get request detail and history when allowed.
`/api/requests/pending`	GET	List the reviewer's assigned pending requests.
`/api/requests/:id/review`	POST	Approve or reject an assigned request.
`/api/health`	GET	Health check; reports `status`, `commit`, and `timestamp`.

4.6.4 Validation and error handling

- Validate input at the API boundary with Zod schemas.
- Reject unknown or malformed fields.
- Parse monetary values carefully; persist as `Decimal(12,2)`.
- Return field-specific `details` when useful.
- Use a central error middleware to map domain errors to the standard response shape.
- Production responses must not expose stack traces, SQL, secrets, or PII.
- Pagination and advanced filtering are not required in Sprint 1; lists are bounded by demo data.

4.7 Authentication and Session Lifecycle

1. **Login:** validate body, find user by email, compare with `bcrypt.compare`, sign a JWT `{ sub, role, iat, exp }` using `JWT_SECRET`, set the `token` cookie, return the user without `passwordHash`.
2. **Authenticated request:** read the cookie, verify signature and expiry, load the user, attach `req.user`. Missing/invalid/expired cookie → `401 UNAUTHORIZED`.
3. **Authorization:** enforce role, ownership, assignment, and state in the order defined in section 4.3.
4. **Current user:** `GET /api/auth/me` returns `{ id, email, name, role }` only.
5. **Logout:** `POST /api/auth/logout` clears the cookie with `maxAge: 0`; the client redirects to `/login`.
6. **Password handling:** bcrypt cost 12; hashes are never logged or returned; demo password is local-only.

4.7.1 Cookie flags

Flag	Value
Name	`token`
`httpOnly`	`true`
`sameSite`	`lax`
`secure`	`true` when `NODE_ENV=production`
`path`	`/`
`maxAge`	8 hours

Sprint 1 uses stateless JWTs. Logout clears the client cookie, but a stolen token remains valid until expiry; server-side revocation is a known limitation and is deferred.

4.8 Configuration, Secrets, and Deployment Design

4.8.1 Environment variables

```
DATABASE_URL=postgresql://user:password@localhost:5434/approval_db
JWT_SECRET=replace-with-long-random-secret
JWT_EXPIRES_IN_SECONDS=28800
PORT=3000
NODE_ENV=development
CLIENT_ORIGIN=http://localhost:5173
COOKIE_SECURE=false
GIT_COMMIT=local
```

- `.env` is ignored by Git.
- `.env.example` documents required variables without real secrets.
- Production secrets are injected by the deployment platform, not committed.
- The database configuration is externalized through `DATABASE_URL`.
- `GIT_COMMIT` is set by CI/CD and returned by `/api/health`, so the running release can be identified from the public URL.

4.8.2 Local development

- PostgreSQL 17 is the target database; run locally or via Docker on host port `5434`.
- Run migrations and seed before starting feature work.
- Run client and server as separate development servers.
- Health endpoint: `GET /api/health`.

4.8.3 Docker and GCP deployment (Lab 5, implemented)

- **One runtime image:** the multi-stage `Dockerfile` installs workspace dependencies, generates the Prisma client, builds the React client and the Express server, and copies only the runtime artifacts into the final stage. The container serves the built client and the API on port `3000`.

- **Local production-like stack:** `docker-compose.yml` runs PostgreSQL 17 (`db`), a one-off `prisma migrate deploy` job (`migrate`), the application (`app`), and an optional `seed` profile. The app starts only after migrations succeed, and database data lives in a named volume.

- **Migrations in the pipeline:** CI runs `prisma migrate deploy` against a service PostgreSQL before tests; deployment runs the same command as a separate Cloud Run job before the service revision is updated. Schema changes therefore follow one path locally, in CI, and in production.

- **Container health check:** the image health check and the post-deploy smoke test call `/api/health`, which returns `status`, `commit`, and `timestamp`.

- **GCP target:** Cloud Run for the application container, Cloud SQL for PostgreSQL, Artifact Registry for images tagged with the commit SHA and deployed by digest, Secret Manager for `DATABASE_URL` and `JWT_SECRET`, and GitHub Actions for CI/CD using Workload Identity Federation.

- **Release identification:** the deploy workflow sets `GIT_COMMIT` to the release SHA, so the running commit is observable from `/api/health` and from the Cloud Run revision.

- **Secrets:** no secret is committed or baked into an image. Local values come from `.env`; cloud values come from Secret Manager and GitHub Actions secrets.

- **Cost controls:** Cloud Run minimum instances `0`, maximum `3`; smallest Cloud SQL tier with storage auto-increase off; resources stopped or removed after grading.

- **Deployment guide:** `docs/labs/lab-05/deployment.md` documents the complete environment variable table, resource names, one-time setup, release procedure, rollback, and evidence mapping.

4.8.4 Reproducibility

- A clean clone must support install, build, test, migration, and seed commands.
- Node and npm versions are documented in the README.
- No real production data is used in local or demo environments.

4.9 Error Handling, Logging, and Security

- A central error handler maps validation and domain errors to the standard error shape.
- Prisma uses parameterized queries, reducing SQL injection risk.
- React escapes rendered text by default; avoid `dangerouslySetInnerHTML`.
- The JSON API plus `sameSite=lax` cookie reduces CSRF risk. Additional CSRF hardening is deferred.
- Rate limiting, audit export, and admin actions are Sprint 3 concerns.
- Logs must not contain passwords, tokens, secrets, or unnecessary personal data.
- Error responses must not reveal stack traces or internal implementation details in production.

4.10 Requirements Traceability

Design area	Requirements
Authentication and session	FR-01, NFR-01, NFR-04
Request creation and validation	FR-02, BR-04, BR-05, NFR-04

Design area	Requirements
My requests and data isolation	FR-03, BR-03
Review queue and decision	FR-04, FR-06, BR-01, BR-02, BR-05, BR-06, BR-07
History and audit write	FR-05, BR-06
Persistence and durability	NFR-03
Testability and traceability	NFR-05
Deployability and configuration	NFR-06

Detailed feature allocation, acceptance criteria, and test IDs are defined in the feature inventory, traceability matrix, and Sprint 1 engineering contracts.

4.11 Known Limitations in Sprint 1

- `ADMIN` role and admin management endpoints are deferred to Sprint 3.
- No attachments, multi-stage approval, or return-for-revision in Sprint 1.
- No real payment, bank, email, or SMS integration.
- Stateless JWT logout clears the cookie but does not revoke an already stolen token.
- Search, filtering, reporting, and analytics are deferred.
- Advanced CSRF protection and rate limiting are deferred unless required by later hardening.

4.12 Version History

Version	Date	Change	Author	Reviewer
v1.0	6 Nov 2026	Initial system-level SDS baseline for Sprint 1.	T	Oum

5 Part D: Three-Sprint Plan

Project: CPE 334 - Workflow and Approval System (Theme 5) • **Deliverable:** Deliverable 1 baseline plan • **Owner:** T (67070501021) • **Reviewer:** Oum (67070501075) • **Related issue:** #2 - System-Level SDS and Three-Sprint Plan • **Status:** v1.0 baseline, reviewed before Sprint 1 feature implementation

5.1 Planning Principles

- Deliver a working vertical slice in every sprint, not isolated layers.
- Keep Sprint 1 scope fixed: authentication, requester flow, reviewer flow, history write, and verification evidence.
- Use engineering contracts and acceptance criteria before implementation.
- Every feature PR is reviewed by the teammate; authors never approve or merge their own PR.
- Later-sprint work stays planned, not silently marked as complete.
- Scope changes require a recorded change request and updated traceability.

5.2 Sprint Overview

Sprint	Target date	Goal	Features
Sprint 1	8 Nov 2026 (Deliverable 1)	Working approval vertical slice with enforced business rules and evidence.	F-01, F-02, F-03, F-04, F-05 (write)
Sprint 2	29 Nov 2026 (Deliverable 2)	Expand audit visibility, revision flow, multi-stage approval, and attachments.	F-05 (view), F-06, F-07, F-08
Sprint 3	13 Dec 2026 (Final)	Search/reporting, admin management, hardening, regression, final release.	F-09, F-10, F-11

5.2.1 Feature dependency graph

```
F-01 -> F-02 -> F-03
F-01/F-02 -> F-04 -> F-05
F-04 -> F-06 -> F-07
F-02 -> F-08
F-03/F-04/F-05 -> F-09
F-01/F-04 -> F-10
F-01..F-10 -> F-11
```

5.3 Sprint 1 - Deliverable 1 (8 Nov 2026)

5.3.1 Goal

Deliver a working vertical slice that supports login, request creation, requester-owned tracking, reviewer assignment, approve/reject decisions, append-only history, and backend enforcement of the Sprint 1 business rules, with automated tests and selected E2E evidence.

5.3.2 Features and scope

Feature ID	Feature	Sprint 1 scope
F-01	Auth and Session	Login, logout, current user, JWT httpOnly cookie, role loading, 401 behavior.
F-02	Request Creation	Validate payload, generate `requestNo`, assign reviewer, persist `PENDING`, write `SUBMITTED` history.
F-03	My Requests and Data Isolation	List own requests, open own detail/history, block cross-user access.
F-04	Review Queue and Decision	Assigned pending queue, approve/reject with comment, state and assignment guards, self-approval guard.
F-05	Approval History	Write append-only history entries for submit/approve/reject. Audit view is Sprint 2.

5.3.3 Dependencies

- F-01 must precede F-02/F-03.
- F-02 must precede F-04.
- Schema and seed foundation from the repository scaffold must be available before F-01.
- Contract E0, E1, and E2 must be committed before their related implementation.

5.3.4 Issue mapping

Issue	Work	Owner	Reviewer	Depends on
#1	Part A/B baseline documents	Oum	T	-
#2	Part C/D SDS and sprint plan	T	Oum	-
#3	F-01 Auth foundation	T	Oum	#2
#4	F-02/F-03 Requester slice	Oum	T	#3
#5	F-04/F-05 Reviewer slice	T	Oum	#3, #4
#6	Part F verification and evidence	Oum + T	Oum + T	#3, #4, #5
#7	Release and Deliverable 1 PDF	Oum + T	Oum + T	#1-#6

5.3.5 Completion outcome

Sprint 1 is complete when:

- AC-AUTH-01..05 and AC-01..AC-12 pass.
- TEST-AUTH-01..05, TEST-01..TEST-13, and selected E2E-01..E2E-03 are implemented and passing on the submitted commit.
- All required Sprint 1 business rules are enforced by the backend, not only by the UI.
- Every acceptance criterion maps to at least one automated test or documented verification.
- The release PR from `sprint-1-staging` to `main` is reviewed and merged.
- Release tag `v0.1.0-sprint1` is created and recorded in the report.
- Parts A-F of the Deliverable 1 PDF are assembled with evidence.

5.3.6 Out of scope

- Attachments (F-08).
- Return-for-revision and resubmission (F-06).
- Multi-stage approval (F-07).
- Search, filtering, and reporting (F-09).
- Admin user/assignment management (F-10).
- Real payment, bank, email, SMS, or external notification integrations.
- Production hardening beyond the Sprint 1 security baseline.

5.3.7 Sprint 1 execution order and parallel work

Critical path:

```
#2 -> #3 -> #4 -> #5 -> #6 -> #7
```

Safe parallel work:

- #1 (Oum) and #2 (T) start in parallel.
- While T implements #3, Oum can finish #1 and draft Contract E1 plus the requester UI shell for #4.
- While Oum implements #4, T can draft Contract E2 and reviewer UI/test scaffolding for #5.
- After #5, both teammates can split #6 evidence collection and #7 PDF assembly.
- Contracts and documentation can be written in parallel, but implementation must wait for the relevant contract and dependencies.

5.4 Sprint 2 - Deliverable 2 (29 Nov 2026)

5.4.1 Goal

Expand the approval workflow beyond the Sprint 1 vertical slice: expose history/audit, add return-for-revision and resubmission, support multi-stage approval, and add controlled attachments.

5.4.2 Features and scope

Feature ID	Feature	Sprint 2 scope
F-05	Approval History and Audit View	Read-only history/audit view built on Sprint 1 append-only records.
F-06	Return for Revision and Resubmission	Reviewer returns a request; requester edits and resubmits under defined state rules.
F-07	Multi-stage Approval	Multiple approval stages with per-stage decision and history.
F-08	Attachments	File upload, storage, authorization, and access control for request attachments.

5.4.3 Dependencies

- F-04 $\rightarrow$ F-06 $\rightarrow$ F-07.
- F-02 $\rightarrow$ F-08.
- F-05 view depends on Sprint 1 history records.
- Contracts, acceptance criteria, and tests must be defined before each feature is implemented.

5.4.4 Completion outcome

- Sprint 2 contracts, acceptance criteria, STS, and DoD are complete.
- Features are integrated, tested, reviewed, and merged into `sprint-1-staging`.
- Updated traceability matrix records actual results, not planned status.
- A deployed candidate is available through the Lab 5 deployment pipeline.
- No Sprint 1 regression is introduced.

5.5 Sprint 3 - Final (13 Dec 2026; progress checkpoint 29 Nov 2026)

5.5.1 Goal

Complete the product and harden it for final release: search/filter/reporting, admin management, security/performance review, regression testing, UAT fixes, and final deployment.

5.5.2 Features and scope

Feature ID	Feature	Sprint 3 scope
F-09	Search, Filter, and Reporting	Search/filter requests and summarize by status, category, and amount.

Feature ID	Feature	Sprint 3 scope
F-10	Admin User and Assignment Management	Admin user/role/reviewer assignment management.
F-11	Hardening, Regression, and Deployment	Security/performance checks, regression, UAT re-test, final release, demo video.

5.5.3 Dependencies

- F-05 $\rightarrow$ F-09.
- F-01/F-04 $\rightarrow$ F-10.
- F-01..F-10 $\rightarrow$ F-11.

5.5.4 Completion outcome

- Final release is merged to `main` and deployed to the planned GCP environment.
- Complete traceability shows actual results for every feature.
- UAT/re-test records and regression results are documented.
- Final PDF and demo video are complete.
- Known limitations, failed/skipped tests, and technical debt are disclosed.

5.6 Exam-Period Finish Plan (1-11 Dec 2026)

- 29 Nov 2026: product should be substantially complete; Sprint 2 is complete and Sprint 3 has meaningful implemented progress.
- 1-11 Dec 2026: bounded fixes, integration, regression, UAT re-test, documentation, and video. Do not reserve main workflows for this period.
- 13 Dec 2026: final release, PDF, and demo video due.

5.6.1 Remaining-work template

Feature/Defe ID	Status	Missing AC/Tests	Owner	Dependenc	Target date	Evidence
Example: F-09	Planned	AC/test not yet written	TBD	F-05	TBD	TBD

Allowed status values: `Planned`, `In Progress`, `Implemented`, `Tested`, `Accepted`.

5.7 Risks and Mitigations

Risk	Impact	Mitigation
Scope creep into multi-stage, attachments, or admin work during Sprint 1	Sprint 1 misses its deadline	Keep Sprint 1 scope fixed; record any change request and re-plan explicitly.
Auth or assignment bugs	Security and business-rule failures	Contract E0/E2, negative tests, teammate review, explicit check order.
E2E flakiness	Unreliable evidence	Use stable seeded data, limit E2E journeys, document retry/reset policy.
PostgreSQL setup differences across machines	Reproduction failures	Docker compose, README commands, migration/seed from a clean clone.
Exam-period schedule compression	Incomplete final release	Freeze scope on 29 Nov, maintain remaining-work list, check progress daily.
Secret leakage	Security and grading failure	`.env` ignored, `.env.example` only, secret scan before submission.
Planned work presented as complete	Loss of credibility and traceability	Mark later-sprint items as planned until they have real AC/test evidence.

5.8 Change Control

Any change to sprint allocation, feature scope, acceptance criteria, or release timing must be recorded with:

Field	Description
Change ID	Unique change identifier.
Date	Date of the change.
Requirement/Feature	Affected FR/BR/NFR/feature.
Change	What changed.
Reason	Why it changed.
Approved by	Both teammates.
Affected AC/TEST/SDS	Impacted artifacts.
Status	Proposed, approved, or rejected.

Planned items must not be silently removed or moved to a later sprint.

5.9 Definition of Done Summary

- Required behavior is implemented and integrated across UI, API, and database.

- Every in-scope acceptance criterion has an automated test or documented verification.
- Required tests are enabled and pass on the submitted commit.
- Teammate review is complete; the author does not self-approve.
- Documentation and traceability match the code.
- Migration and seed run from a clean database.
- No real secrets or production data are committed.
- Known limitations and incomplete work are disclosed honestly.

5.10 Version History

Version	Date	Change	Author	Reviewer
v1.0	6 Nov 2026	Initial three-sprint plan baseline.	T	Oum

6 Part E.0: Feature Contract E0 (F-01 Auth Foundation)

Feature ID: F-01 • **Issue:** #3 - Auth foundation: schema, seed, login/logout/me • **Linked requirements:** FR-01, NFR-01, NFR-04 • **Owner:** T (67070501021) • **Reviewer:** Oum (67070501075) • **Status:** v1.1 implemented and verified locally • **Artifact:** *docs/sprints/sprint-1/feature-00-auth.md*

6.1 Feature SDS

The system must support authenticated sessions for the requester and reviewer slices:

- verify email/password credentials against the `User` table;
- issue a signed JWT in an httpOnly cookie;
- load the current user for protected requests;
- expose login, logout, and current-user endpoints;
- reject missing, invalid, expired, or deleted-user sessions with `401`;
- reject authenticated users who lack the required role with `403`;
- never return or log `passwordHash`.

Authentication is stateless in Sprint 1. Logout clears the cookie on the client; server-side token revocation is a documented limitation and is deferred.

6.2 UI Specification

6.2.1 LoginPage

- Fields: `email`, `password`.
- Inline validation for required fields and email format.
- Loading state while the request is in flight.
- Invalid credentials show the server error code/message.
- Successful login redirects according to the user's role.
- The client must not store the JWT in `localStorage`.

6.2.2 Session behavior

- Authenticated screens call `GET /api/auth/me` on load or use the login response.
- A `401` response clears client auth state and redirects to `/login`.
- The logout action calls `POST /api/auth/logout` and redirects to `/login`.
- Role-aware navigation is a UX aid, not an authorization boundary.

6.3 API Specification

All responses use the standard envelope:

- Success: `{ "data": ... }`

- Failure: `{ "error": { "code": "...", "message": "...", "details"?: ... } }`

Endpoint	Method	Request	Success	Errors
`/api/auth/login`	POST	`{ email, password }`	`200 { data:{ user:{ id, email, name, role } } }` + `Set-Cookie`	`400 VALIDATION_ERROR`; `401 UNAUTHORIZED`
`/api/auth/logout`	POST	cookie	`200 { data:{ message } }` + cleared cookie	`401 UNAUTHORIZED`
`/api/auth/me`	GET	cookie	`200 { data:{ user } }`	`401 UNAUTHORIZED`

6.3.1 Cookie contract

Attribute	Value
Name	`token`
`httpOnly`	`true`
`sameSite`	`lax`
`secure`	`true` in production; `false` for local/test
`path`	`/`
`maxAge`	8 hours

6.3.2 Authentication header behavior

- The API reads the cookie, not an `Authorization` header.
- Missing or invalid cookie → `401 UNAUTHORIZED`.
- Expired token → `401 UNAUTHORIZED`.
- A token whose user no longer exists → `401 UNAUTHORIZED`.

6.3.3 Role guard behavior

`requireRole('REVIEWER')` returns `403 FORBIDDEN` for an authenticated `REQUESTER`. For Issue #3, `TEST-AUTH-04` exercises this middleware through a test-only reviewer route; the product reviewer endpoints are added in Issue #5.

6.4 Data Specification

Uses the existing Prisma models:

- `User.id` (UUID)
- `User.email` (unique)
- `User.passwordHash` (bcrypt, cost 12)
- `User.name`

- `User.role` (`REQUESTER`, `REVIEWER`, future `ADMIN`)
- `User.createdAt`

The seed script is idempotent and upserts three synthetic accounts. No schema change beyond the foundation is required for this contract.

6.5 Acceptance Criteria

ID	Acceptance criterion
AC-AUTH-01	Valid email/password login returns `200`, sets an httpOnly `token` cookie, and returns the user without `passwordHash`.
AC-AUTH-02	Wrong password or unknown email returns `401 UNAUTHORIZED` and does not set a cookie.
AC-AUTH-03	`GET /api/auth/me` returns the current user when authenticated; missing/invalid session returns `401`; logout clears the cookie and subsequent `/me` returns `401`.
AC-AUTH-04	A `REQUESTER` calling a reviewer-only route receives `403 FORBIDDEN`; backend role enforcement is not UI-only.
AC-AUTH-05	Stored passwords use bcrypt with cost 12; JWT expiry is 8 hours; no plaintext password or hash appears in responses.

6.6 Software Test Specification

Test ID	AC	Level	Preconditions / input	Action	Expected
TEST-AUTH-01	AC-AUTH-01	API integration	Seeded active user	`POST /api/auth/login` with valid credentials	`200`; `Set-Cookie` contains httpOnly `token`; body contains public user fields; no `passwordHash`
TEST-AUTH-02	AC-AUTH-02	API integration	Wrong password and unknown email	`POST /api/auth/login`	`401 UNAUTHORIZED`; no `Set-Cookie`
TEST-AUTH-03	AC-AUTH-03	API integration	No cookie; valid session; logout	`GET /me`; `POST /logout`; `GET /me` again	`401`; then `200`; then cleared cookie and `401`
TEST-AUTH-04	AC-AUTH-04	API integration	Requester cookie and reviewer cookie	Request a reviewer-only guard route	Requester `403 FORBIDDEN`; reviewer `200`
TEST-AUTH-05	AC-AUTH-05	Unit/API integration	DB user row and login response	Inspect hash and decode JWT	Hash starts with `$2` and cost 12; `exp -iat = 8h`; no plaintext password

Additional negative checks:

- invalid email shape → `400 VALIDATION_ERROR`;

- missing body fields → `400 VALIDATION_ERROR`;
- logout without a session → `401 UNAUTHORIZED`;
- tampered token → `401 UNAUTHORIZED`.

6.7 Definition of Done

x Contract committed before implementation.

x Migration and seed run from a clean database (`prisma migrate reset` + seed).

x Login/logout/me implementation uses the standard response shape.

x Cookie flags match this contract.

x `TEST-AUTH-01..05` implemented and passing (9 server tests total).

x No plaintext password or hash in responses/logs.

Teammate review completed; author does not self-approve. (pending PR review)

PR merged into `sprint-1-staging`. (pending)

x Red/green TDD evidence recorded (red run: `8 failed | 1 passed`).

6.8 Version History

Version	Date	Change	Reason	Commit/PR
v1.0	10 Nov 2026	Initial auth contract.	Sprint 1 baseline.	`fc8f548`
v1.1	10 Nov 2026	Implementation notes: config/env loader, Prisma singleton, central error handler, `requireAuth`/`requireRole`, auth routes, initial migration, TEST-AUTH-01..05 green.	Auth foundation implemented and verified.	PR #12 (`ad10d0b`)
v1.2	11 Nov 2026	Prisma root scripts now load the repository-root `.env`; seed loads it explicitly. Verified `cp .env.example .env && npm run prisma:migrate && npm run prisma:seed` in a clean shell.	Review finding: clean-clone setup failed with `P1012 Environment variable not found:DATABASE_URL`.	PR #12 (`ad10d0b`)

7 Part E.1: Feature Contract E1 (F-02/F-03 Requester Slice)

Feature IDs: F-02, F-03 • **Issue:** #4 - Requester slice: create request, my requests, data isolation • **Linked requirements:** FR-02, FR-03, BR-03, BR-04, BR-05, BR-06, NFR-03, NFR-04 • **Owner:** Oum (67070501075) • **Reviewer:** T (67070501021) • **Status:** v1.1 implemented and verified in CI (5 test files / 20 tests passed) • **Artifact:** *docs/sprints/sprint-1/feature-01.md*

7.1 Feature SDS

The requester vertical slice allows authenticated users (`REQUESTER` and `REVIEWER`, per D-02) to create expense/purchase approval requests, persist them with an initial `SUBMITTED` history record, and retrieve only the requests they own (or are assigned to review).

7.1.1 Workflow & Business Rules

1. **Creation & Validation (`F-02` / `BR-04`):**

- Validate request payload on the server using Zod (`.strict()`).
- `title`: trimmed string, 3–150 characters.
- `amount`: positive number > 0 and <= `9,999,999,999.99` (stored as `Decimal(12,2)`).
- `category`: must be one of `EQUIPMENT | TRAVEL | SOFTWARE | SUPPLIES`.
- `description`: trimmed string, 1–1,000 characters.
- Invalid payloads return `400 VALIDATION_ERROR` with field details and persist nothing.

1. **Request Number Generation (`D-10`):**

- Server generates `requestNo` in format `REQ-YYYYMMDD-NNNN` (e.g. `REQ-20261111-0001`), retrying on unique-constraint collision.

1. **Deterministic Reviewer Assignment (`BR-05` / `D-03`):**

- `EQUIPMENT` or `TRAVEL` → assigned to `t.reviewer@cpe334.local`.
- `SOFTWARE` or `SUPPLIES` → assigned to `manager.hybrid@cpe334.local`.
- If the requester is the mapped reviewer for that category, the system assigns the alternate reviewer instead.

1. **Atomic Creation (`BR-06` / `D-14`):**

- Inside `prisma.$transaction`: insert `ApprovalRequest` with `status = PENDING` and insert `ApprovalHistory` with `action = SUBMITTED` and `actorId = currentUser.id`.

1. **My Requests & Data Isolation (`F-03` / `BR-03`):**

- `GET /api/requests/my` returns only records where `requesterId = currentUser.id`, ordered by `createdAt DESC`.
- `GET /api/requests/:id` returns request details and ordered histories only if `requesterId === currentUser.id` or `assignedReviewerId === currentUser.id`; otherwise returns `403 ACCESS_DENIED` without leaking any private request fields.

7.2 UI Specification

7.2.1 Screens & Routes

Route	Screen	Access	Key States
/login	LoginPage	Public	email/password form, inline validation, loading, error banner, role redirect
/requests/new	NewRequestPage	Authenticated	fields (title, amount, category, description), validation, submitting disabled state, API error banner, redirect to detail/list on 201
/requests/my	MyRequestsPage	Authenticated	loading state, empty state with CTA, table (requestNo, title, amount, category, status, assignedReviewer, createdAt), error banner
/requests/:id	RequestDetailPage	Owner or assigned reviewer	loading, 404 not found, 403 forbidden state, full request fields, chronological approval history timeline

7.2.2 Shared UI Conventions

- Client-side validation mirrors server rules (title 3–150, amount > 0, category select, description 1–1000).
- Submit button disabled while in flight; no optimistic status mutation.
- Navigation bar displays current user name/role and Logout button (POST /api/auth/logout).
- Never expose passwordHash or token values.

7.3 API Specification

All responses use the standard envelope:

- Success: { "data": ... }
- Failure: { "error": { "code": "...", "message": "...", "details"?: ... } }

Endpoint	Method	Request	Success	Errors
/api/requests	POST	{ title, amount, category, description }	201 { data:{ request } }	400 VALIDATION_ERROR; 401 UNAUTHORIZED; 403 FORBIDDEN
/api/requests/my	GET	cookie	200 { data:{ requests:[...] } }	401 UNAUTHORIZED
/api/requests/:id	GET	cookie	200 { data:{ request, histories } }	401 UNAUTHORIZED; 403 ACCESS_DENIED; 404 NOT_FOUND

7.3.1 Response Field Notes

- `amount` is serialized as a 2-decimal string (e.g., `"1500.00"`) to preserve `Decimal(12,2)` precision.
- `requester`, `assignedReviewer`, and `actor` nested objects expose only `{ id, name, email, role }` (never `passwordHash`).
- `histories` are ordered chronologically (`createdAt ASC`).

7.4 Data Specification

Uses existing Prisma models from the foundation schema (`20261110084745_init_schema`):

- `ApprovalRequest`: `id`, `requestNo` (unique), `title`, `amount` (`Decimal(12,2)`), `category`, `description`, `status` (`PENDING`), `requesterId`, `assignedReviewerId`, `createdAt`, `updatedAt`.
- `ApprovalHistory`: `id`, `requestId`, `actorId`, `action` (`SUBMITTED`), `comment`, `createdAt`.
- Indexes utilized: `ApprovalRequest_requesterId_createdAt_idx` and `ApprovalHistory_requestId_createdAt_idx`.
- No new database migration is required for Contract E1.

7.5 Acceptance Criteria

ID	Acceptance Criterion
AC-01	Authenticated `REQUESTER` (or `REVIEWER`) submitting valid request fields receives `201` with `status = PENDING`, a unique `requestNo`, a deterministic `assignedReviewerId` per BR-05, and an initial `SUBMITTED` history entry.
AC-02	Submitting invalid payloads (`amount <= 0`, `amount > 9999999999.99`, trimmed `title` < 3 or > 150 chars, unknown `category`, empty or > 1000 char `description`, unknown body keys) returns `400 VALIDATION_ERROR` and creates no database record.
AC-03	`GET /api/requests/my` returns `200` with only the authenticated user's own requests, ordered newest first (`createdAt DESC`).
AC-04	`GET /api/requests/:id` returns `403 ACCESS_DENIED` without private data when requested by a user who is neither the requester nor the assigned reviewer; unauthenticated requests to any `/api/requests*` route return `401 UNAUTHORIZED`.
AC-04b	Created requests and their `SUBMITTED` history entries persist in PostgreSQL across process/client restarts.

7.6 Software Test Specification (STS)

Test ID	AC	Level	Preconditions / Input	Action	Expected Result
TEST-01	AC-01	API/DB integration	Authenticated requester + valid `EQUIPMENT` payload	`POST /api/requests`	`201`; `status = PENDING`; `requestNo` matches `REQ-YYYYMMDD-NNNN`; `assignedReviewerId` is `t.reviewer`; `SUBMITTED` history row exists in DB
TEST-02	AC-02	API integration (boundary/negative)	`amount` 0 / -10 / `10000000000`; `title` 2/151c; bad `category`; empty `desc`	`POST /api/requests`	`400 VALIDATION_ERROR`; DB request count unchanged
TEST-03	AC-03	API integration	User A has 2 requests, User B has 1 request	`GET /api/requests/my` as User A	`200`; returns only A's 2 requests, ordered newest first
TEST-04	AC-04	API integration	Request owned by User A (assigned to Reviewer R1); User B and unrelated Reviewer R2 try to access	`GET /api/requests/:id` as B / R2 / anonymous	Anonymous → `401 UNAUTHORIZED`; User B & R2 → `403 ACCESS_DENIED` with no private data
E2E-01	AC-01, AC-03, AC-04b	E2E (Playwright)	Seeded requester account	Login → navigate to `/requests/new` → submit valid request → view `/requests/my` → open detail → reload page	Persisted request and `SUBMITTED` history visible after page reload

Additional boundary & unit checks:

- Reviewer creating a request in their own primary category (e.g., `t.reviewer` creating `EQUIPMENT`) automatically gets assigned to the alternate reviewer (`manager.hybrid`) so `assignedReviewerId !== requesterId`.
- Unknown request UUID returns `404 NOT_FOUND`.
- Client component tests verify form validation messages, loading/disabled submit state, and empty/error states in `MyRequestsPage`.

7.7 Definition of Done

x Contract E1 (`v1.0`) committed before feature tests and implementation (`bb4251b`).

x Automated API/DB tests (`TEST-01..TEST-04`, 6 server tests) and client component tests (`App.test.tsx`, `Requests.test.tsx`, 4 tests) implemented and passing in CI (`43047999614`: 5 files / 20 tests passed).

x Playwright `E2E-01` spec (`e2e/requester.spec.ts`) implemented (to be executed against local running stack when Docker is up).

x TDD commit sequence visible (`bb4251b` contract → `d80cc44` tests → `331294f` implementation → `37a343b` workspace test fix).

x All responses conform to `{ data }` / `{ error: { code, message, details? } }` and leak no `passwordHash`.

x Persistence verified against PostgreSQL in CI (`requests.test.ts` queries `ApprovalRequest` and `ApprovalHistory` directly via Prisma).

Teammate review completed on PR #15 targeting `sprint-1-staging` (pending T's review).

7.8 Version History

Version	Date	Change	Reason	Commit/PR
v1.0	11 Nov 2026	Initial Contract E1 specification (SDS, UI, API, AC, STS, DoD).	Sprint 1 pre-implementation contract for Issue #4.	`bb4251b`
v1.1	11 Nov 2026	Implementation notes: `request.service.ts`, `routes/requests.ts`, React requester UI (`LoginPage`, `NewRequestPage`, `MyRequestsPage`, `RequestDetailPage`), `vitest.workspace.ts`, `E2E-01` spec, and CI verification (20 tests green).	Requester slice implemented and verified.	PR #15 (`fd0c875`)

8 Part E.2: Feature Contract E2 (F-04/F-05 Reviewer Slice)

Feature IDs: F-04, F-05 • **Issue:** #5 - Reviewer slice: assigned queue, decision, history, self-approval guard • **Linked requirements:** FR-04, FR-05, FR-06, BR-01, BR-02, BR-05, BR-06, BR-07, NFR-01, NFR-04 • **Owner:** T (67070501021) • **Reviewer:** Oum (67070501075) • **Status:** v1.1 implemented and verified locally (PR pending review) • **Artifact:** *docs/sprints/sprint-1/feature-02.md*

8.1 Feature SDS

The reviewer slice lets a `REVIEWER` see only the `PENDING` requests assigned to them, inspect one request with its history, and record an `APPROVE` or `REJECT` decision with an optional/required comment. A decision changes the request status and appends an `ApprovalHistory` row inside one database transaction. The backend enforces every business rule; hiding UI controls is not authorization.

8.1.1 Workflow and enforcement order

`POST /api/requests/:id/review` checks are executed in this exact order:

1. **Authentication** - no/invalid `token` cookie → `401 UNAUTHORIZED`.
2. **Role** - role other than `REVIEWER` → `403 FORBIDDEN`.
3. **Existence** - unknown request id → `404 NOT_FOUND`.
4. **No self-approval (BR-01)** - `requesterId === currentUser.id` → `403 SELF_APPROVAL_NOT_ALLOWED`.
5. **Assignment (BR-05)** - `assignedReviewerId !== currentUser.id` → `403 ACCESS_DENIED`.
6. **State lifecycle (BR-02)** - status is not `PENDING` → `400 INVALID_STATE_TRANSITION`.
7. **Decision payload (BR-07)** - invalid `decision`/`comment` → `400 VALIDATION_ERROR`.
8. **Commit** - status update + `ApprovalHistory` insert in one transaction (BR-06).

Checking self-approval before assignment makes BR-01 testable even when the owner's request is assigned to another reviewer.

8.1.2 Rules

- `GET /api/requests/pending` returns only requests where `assignedReviewerId = currentUser.id` and `status = PENDING`, oldest first (`createdAt ASC`).
- `APPROVE` sets `APPROVED`; `REJECT` sets `REJECTED`; no other transitions are allowed.
- `REJECT` requires a trimmed comment of 3-500 characters.
- `APPROVE` accepts no comment or a trimmed comment of at most 500 characters. Empty/whitespace-only comments are stored as `null`.
- The status update uses a conditional write (`where: { id, status: PENDING }`); if the request was decided concurrently, the transaction fails with `INVALID_STATE_TRANSITION` and no history row is added.
- History rows are append-only. Sprint 1 exposes no update or delete history endpoint.

8.2 UI Specification

8.2.1 Routes and screens

Route	Screen	Access	Key states
`/reviewer/pending`	`PendingReviewsPage`	`REVIEWER`	loading, empty (”no assigned pending requests”), list, error banner with `code:message`
`/reviewer/requests/:id`	`ReviewDetailPage`	`REVIEWER`	loading, 404 not found, 403 forbidden, request fields, chronological history, decision form, submitting/disabled, success state, error banner

8.2.2 PendingReviewsPage

- Table columns: `requestNo`, `title`, `requester`, `category`, `amount`, `createdAt`, `action`.
- Amount rendered as a 2-decimal THB value.
- ”Review” link navigates to `/reviewer/requests/:id`.
- Empty state explains that only requests assigned by category appear here.

8.2.3 ReviewDetailPage

- Shows the same request fields as the requester detail page plus the assigned reviewer and history timeline.
- When the request is `PENDING`, a decision form is shown with `APPROVE`/`REJECT` options and a comment textarea.
- When the current user is the owner, a warning explains that self-approval is forbidden and the server will reject the attempt (`SELF_APPROVAL_NOT_ALLOWED`). The form remains submit-capable so the backend rule can be demonstrated and tested; the UI never silently bypasses it.
- When the current user is not the assigned reviewer, a warning explains that the request is not assigned to them (`ACCESS_DENIED` from the server).
- Client validation mirrors the server for `REJECT` comments (3-500 characters) and `APPROVE` comments (max 500).
- On success the page re-renders the updated status and appends the new history row; no optimistic status change.
- After a successful decision, the request no longer appears in the pending queue.

8.2.4 Navigation

- `LoginPage` sends `REVIEWER` users to `/reviewer/pending` and `REQUESTER` users to `/requests/my` after login.
- `Layout` shows the `Pending Reviews` link only for `REVIEWER` users (existing behavior).

8.3 API Specification

All responses use the standard envelope:

- Success: `{ "data": ... }`

- Failure: `{ "error": { "code": "...", "message": "...", "details"?: ... } }`

Endpoint	Method	Request	Success	Errors
`/api/requests/pending`	GET	cookie	`200 { data:{ requests:[...] } }`	`401 UNAUTHORIZED`; `403 FORBIDDEN`
`/api/requests/:id/review`	POST	`{ decision: "APPROVE" | "REJECT", comment?: string }`	`200 { data:{ request, history } }`	

8.3.1 Response field notes

- `request` uses the same serialization as Contract E1: `amount` is a 2-decimal string; nested `requester`/`assignedReviewer` expose only `{ id, name, email, role }`.
- `history` is the newly appended row with `{ id, requestId, actorId, actor, action, comment, createdAt }`.
- `GET /api/requests/pending` items include the requester public profile and are ordered by `createdAt ASC`.

8.4 Data Specification

Uses the existing schema from migration `20261110084745_init_schema`; no new migration is required.

- `ApprovalRequest.status`: `PENDING` → `APPROVED` or `REJECTED`.
- `ApprovalRequest.updatedAt` changes automatically on update.
- `ApprovalHistory` insert: `requestId`, `actorId = currentUser.id`, `action = APPROVED | REJECTED`, `comment = trimmed comment or null`.
- The status update and history insert run inside `prisma.$transaction`.
- The conditional update (`where: { id, status: PENDING }`) protects against double decisions and concurrent reviewers.
- History is append-only; Sprint 1 has no update/delete endpoint for `ApprovalHistory`.

8.5 Acceptance Criteria

ID	Acceptance Criterion
AC-05	`GET /api/requests/pending` returns `200` with only `PENDING` requests where `assignedReviewerId = currentUser.id`; requests assigned to another reviewer and already-decided requests are excluded. Non-reviewers receive `403 FORBIDDEN`; unauthenticated requests receive `401 UNAUTHORIZED`.
AC-06	The assigned reviewer submits `APPROVE` or `REJECT` with a comment and receives `200`; the request status changes to `APPROVED`/`REJECTED`, `updatedAt` changes, and exactly one new `ApprovalHistory` row is appended with the actor, action, and comment.

ID	Acceptance Criterion
AC-07	A `REVIEWER` who owns the request cannot approve or reject it, even when it is assigned to another reviewer, and receives `403 SELF_APPROVAL_NOT_ALLOWED`; the status and history do not change. The check runs before the assignment check.
AC-08	Approving or rejecting a request that is already `APPROVED` or `REJECTED` returns `400 INVALID_STATE_TRANSITION` and appends no history.
AC-09	A `REVIEWER` who is not `assignedReviewerId` receives `403 ACCESS_DENIED` and cannot change the request.
AC-10	A `REQUESTER` (no `REVIEWER` role) calling either reviewer endpoint receives `403 FORBIDDEN` from the backend, independent of the UI.
AC-11	History is append-only: decisions add new rows, existing rows are never modified or deleted, no update/delete endpoint exists, and rows persist after an application process restart.
AC-12	`decision` outside 'APPROVE

8.6 Software Test Specification (STS)

Test ID	AC	Level	Preconditions / Input	Action	Expected Result
TEST-05	AC-05	API/DB integration	Pending requests assigned to R1 and R2; one decided request assigned to R1	`GET /api/requests/pending` as R1	`200`; only R1's `PENDING` request; excludes R2's and the decided request; requester profile included
TEST-06	AC-06	API/DB integration	Assigned `PENDING` request	`POST /review` with `APPROVE` + comment	`200`; `APPROVED`; `updatedAt` changed; one `APPROVED` history row with actor/comment
TEST-07	AC-06	API/DB integration	Assigned `PENDING` request	`POST /review` with `REJECT` + 3-500 char comment	`200`; `REJECTED`; one `REJECTED` history row with comment
TEST-08	AC-07	API/DB integration	`manager.hybrid` (REVIEWER) owns an `EQUIPMENT` request assigned to `t.reviewer`	`POST /review` as `manager.hybrid`	`403 SELF_APPROVAL_NOT_ALLOWED`; status remains `PENDING`; history unchanged
TEST-09	AC-08	API/DB integration	Request already `APPROVED`	`POST /review` again	`400 INVALID_STATE_TRANSITION`; no new history
TEST-10	AC-10	API integration	`REQUESTER` cookie	`GET /pending` and `POST /review`	`403 FORBIDDEN` on both
TEST-11	AC-09	API integration	`manager.hybrid` reviews a request assigned to `t.reviewer`	`POST /review`	`403 ACCESS_DENIED`; no change

Test ID	AC	Level	Preconditions / Input	Action	Expected Result
TEST-12	AC-12	API integration	Bad `decision`; `REJECT` with no/2/501-char comment; `APPROVE` with 501-char comment	`POST /review`	`400 VALIDATION_ERROR` for each; status/history unchanged
TEST-13	AC-11	API/DB integration	Request with `SUBMITTED` history; approve it; restart the application	Inspect history rows before/after; attempt `PUT`/`DELETE` on history	Existing row unchanged; new row appended; no update/delete route; rows persist after restart
E2E-02	AC-06	E2E (Playwright)	Request created by a requester and assigned to `t.reviewer`	Login as reviewer → pending queue → open request → `APPROVE` + comment	Success state; status `APPROVED`; `APPROVED` history visible; request leaves the pending queue
E2E-03	AC-07	E2E (Playwright)	`manager.hybrid` owns a request	Login as `manager.hybrid` → open `/reviewer/requests/:id` → attempt `APPROVE`	Error banner contains `SELF_APPROVAL_NOT_ALLOWED`; status remains `PENDING`

Additional negative checks:

- Unauthenticated `GET /api/requests/pending` and `POST /api/requests/:id/review` → `401 UNAUTHORIZED`.
- Unknown request id → `404 NOT_FOUND`.
- `REJECT` comment is trimmed before the 3-500 length check.

8.7 Definition of Done

x Contract E2 (`v1.0`) committed before feature tests and implementation (`4d3731a`).

x `review.service.ts` implemented with the exact check order in §1.

x PendingReviewsPage and ReviewDetailPage implemented with loading/empty/error/forbidden states.

x `TEST-05..TEST-13` implemented and passing (11 server tests); 3 client component tests added.

x `E2E-02` and `E2E-03` implemented and executed against the local stack (Playwright, 2 passed).

x History is append-only and survives process restart (verified: `SUBMITTED,APPROVED` retained after restart).

x All responses use `{ data }` / `{ error: { code, message, details? } }` and leak no `passwordHash`.

Teammate review completed on the PR targeting `sprint-1-staging`; author does not self-approve.

x Contract version history and sprint README updated.

8.8 Implementation Notes

Area	Artifact	Notes
Queue + decision service	`server/src/services/review.service.ts`	`listPendingReviews` (assigned + `PENDING`, `createdAt ASC`) and `submitReview` (BR-01 → BR-05 → BR-02 → BR-07 → transaction).

Area	Artifact	Notes
Routes	`server/src/routes/requests.ts`	`GET /api/requests/pending` and `POST /api/requests/:id/review` behind `requireRole(Role.REVIEWER)`.
Reviewer UI	`client/src/pages/PendingReviewsPage.tsx`, `client/src/pages/ReviewDetailPage.tsx`	Queue table; detail with decision form, owner/unassigned warnings, client-side comment validation, no optimistic status change.
Routing	`client/src/App.tsx`, `client/src/pages/LoginPage.tsx`	`/reviewer/pending` and `/reviewer/requests/:id` wired; `REVIEWER` login redirects to the queue.
Tests	`server/tests/review.test.ts`, `client/src/pages/Reviews.test.tsx`, `e2e/reviewer.spec.ts`	Red-phase commit `2c74f2c`; green after implementation.
Test isolation	`vitest.config.ts`	Root projects now run with `fileParallelism: false` because every project shares one PostgreSQL database, which made the requester suite's global row-count assertion race.

8.9 Version History

Version	Date	Change	Reason	Commit/PR
v1.0	12 Nov 2026	Initial Contract E2 specification (SDS, UI, API, AC-05..AC-12, STS, DoD).	Sprint 1 pre-implementation contract for Issue #5.	`4d3731a`
v1.1	12 Nov 2026	Marked implemented: backend service/routes, reviewer UI, TEST-05..13, E2E-02/03, restart check, implementation notes, test-isolation note.	Reviewer slice implemented and verified.	PR #16 (`2bcbf4a`)

9 Part F: Working Sprint 1 Verification Evidence Package

Deliverable: CPE 334 Group Project Deliverable 1 — Part F • **Project:** Corporate Expense & Purchase Request Approval System (*ssiriwan/cpe334-approval-system*) • **Issue:** #6 — *[Part F] Sprint 1 verification and evidence package* • **Verified Commit:** *2bcbf4a* (*sprint-1-staging*, post-PR #16 merge) • **Authors:** Oum (67070501075, *@ssiriwan*) & T (67070501021, *@thhanabun*) • **Verification Date:** 2026-11-12 (re-verified 2026-11-13 by T on a clean database)

9.1 Setup Summary

- **Architecture:** TypeScript monorepo (`client/` React + Vite, `server/` Express + Prisma ORM, `prisma/` PostgreSQL 17 schema & seed).
- **Local Verification Stack:**
- PostgreSQL 17 running in Docker (`localhost:5434`, database `approval_db`)
- Schema & Seed: `npx prisma migrate deploy --schema=prisma/schema.prisma` + `npm run prisma:seed` (3 demo accounts: `oum.requester@cpe334.local`, `t.reviewer@cpe334.local`, `manager.hybrid@cpe334.local`)
- Vitest Workspace (`vitest.config.ts` with `fileParallelism: false` to prevent shared-DB race conditions)
- Playwright Chromium (`e2e/*.spec.ts`) starting `npm run dev:server` (`:3000`) and `npm run dev:client` (`:5173`)

9.2 Build & Automated Test Results (Commit `2bcbf4a`)

Check	Command	Result	Duration
DB Migration & Seed	`npx prisma migrate deploy --schema=prisma/schema.prisma && npm run prisma:seed`	**Pass** (1 migration applied, 3 demo accounts seeded)	-
Unit / API / DB Suite	`npx vitest run --config vitest.config.ts --reporter=verbose`	7 files passed (7)	34 tests passed (34)
Production Build	`npm run build` (`tsc -b && vite build` + `tsc -p tsconfig.json`)	**Pass** (client bundle + server JS compiled cleanly)	2.94s+
E2E Suite (Chromium)	`npx playwright test --reporter=list`	3 passed (3)	0 failed
GitHub Actions CI	Workflow `CI and Docker` (Run 43193876219 (https://github.com/ssiriwan/cpe334-approval-system/actions/runs/43193876219))	**Success** (migrate, seed, 34 tests, build, Docker build & `/api/health` smoke test)	1m49s
Re-verification by reviewer (T)	`npx prisma migrate deploy` + `npm run prisma:seed` + `npm test` on a fresh PostgreSQL 17 volume	**Pass** (1 migration applied, 3 demo accounts seeded, **7 files / 34 tests passed**)	79.86s

9.3 Acceptance Criteria → Test Traceability Matrix

Every Sprint 1 Acceptance Criterion (`AC-AUTH-01..05` and `AC-01..12`) is mapped to executed automated tests on commit `2bcbf4a`:

AC ID	Feature / Rule	Test ID	Test File & Case Name	Result	Commit
AC-AUTH-01	F-01 Login	`TEST-AUTH-01`	`server/tests/auth.test.ts — TEST-AUTH-01:logs in with valid credentials and sets an httpOnly cookie`	Pass	`2bcbf4a`
AC-AUTH-02	F-01 Invalid Login	`TEST-AUTH-02`	`server/tests/auth.test.ts — TEST-AUTH-02:rejects wrong passwords and unknown users without setting a cookie`	Pass	`2bcbf4a`
AC-AUTH-03	F-01 Session / Logout	`TEST-AUTH-03`	`server/tests/auth.test.ts — TEST-AUTH-03: rejects unauth /me, returns user, clears session`	Pass	`2bcbf4a`
AC-AUTH-04	F-01 Role Guard	`TEST-AUTH-04`	`server/tests/auth.test.ts — TEST-AUTH-04:allows REVIEWER and rejects REQUESTER on a reviewer-only route`	Pass	`2bcbf4a`

AC ID	Feature / Rule	Test ID	Test File & Case Name	Result	Commit
AC-AUTH-05	F-01 / NFR-01 Security	TEST-AUTH-05	server/tests/auth.test.ts — TEST-AUTH-05:stores bcrypt cost 12 and issues an 8-hour JWT without leaking secrets	Pass	2bcbf4a
AC-01	F-02 / BR-05 Create & Assign	TEST-01	server/tests/requests.test.ts — TEST-01: creates a PENDING request with requestNo, assigned reviewer, and SUBMITTED history (+ E2E-01)	Pass	2bcbf4a
AC-02	F-02 / BR-04 Validation	TEST-02	server/tests/requests.test.ts — TEST-02: rejects invalid payloads and boundary violations without persisting records	Pass	2bcbf4a

AC ID	Feature / Rule	Test ID	Test File & Case Name	Result	Commit
AC-03	F-03 My Requests	TEST-03	server/tests/requests.test.ts — TEST-03: returns only the authenticated user requests ordered newest first	Pass	2bcbf4a
AC-04	F-03 / BR-03 Isolation & Persistence	TEST-04, E2E-01	server/tests/requests.test.ts — TEST-04: enforces data isolation on GET /api/requests/:id; e2e/requester.spec.ts — E2E-01	Pass	2bcbf4a
AC-05	F-04 / BR-05 Pending Queue	TEST-05	server/tests/review.test.ts — TEST-05: returns only PENDING requests assigned to the current reviewer	Pass	2bcbf4a
AC-06	F-04 / F-05 Approve & Reject	TEST-06, TEST-07, E2E-02	server/tests/review.test.ts — TEST-06, TEST-07; e2e/reviewer.spec.ts — E2E-02	Pass	2bcbf4a

AC ID	Feature / Rule	Test ID	Test File & Case Name	Result	Commit
AC-07	F-04 / BR-01 No Self-Approval	TEST-08, E2E-03	server/tests/review.test.ts — TEST-08: blocks self-approval before the assignment check (BR-01); e2e/reviewer.spec.ts — E2E-03	Pass	2bcbf4a
AC-08	F-04 / BR-02 Lifecycle	TEST-09	server/tests/review.test.ts — TEST-09: rejects a second decision with INVALID_STATE_-TRANSITION and appends no history	Pass	2bcbf4a
AC-09	F-04 / BR-05 Assigned Only	TEST-11	server/tests/review.test.ts — TEST-11: rejects an unrelated reviewer with ACCESS_DENIED	Pass	2bcbf4a
AC-10	F-04 Role Enforcement	TEST-10	server/tests/review.test.ts — TEST-10: rejects a REQUESTER calling the review endpoint with FORBIDDEN	Pass	2bcbf4a

AC ID	Feature / Rule	Test ID	Test File & Case Name	Result	Commit
AC-11	F-05 / BR-06 Append-Only History	TEST-13	server/tests/review.test.ts — TEST-13:keeps history append-only and exposes no update/delete route (+ manual restart check)	Pass	2bcbf4a
AC-12	F-04 / BR-07 Decision Payload	TEST-12	server/tests/review.test.ts — TEST-12: validates the decision payload without changing the request	Pass	2bcbf4a

Additional automated checks passing on *2bcbf4a*: *server/tests/health.test.ts* (2/2: */api/health* 200 and */api/* 404 JSON envelope), *client/src/App.test.tsx* (1/1), *client/src/pages/Requests.test.tsx* (3/3), *client/src/pages/Reviews.test.tsx* (3/3).

9.4 Three Demonstration Scenarios

9.4.1 Scenario 1: Normal End-to-End Journey (UI → API → DB → Retrieval)

- **Feature / AC IDs:** F-01, F-02, F-03, F-04, F-05 (AC-01, AC-03, AC-04, AC-05, AC-06)
- **Roles:** Oum Requester (REQUESTER), T Reviewer (REVIEWER)
- **Preconditions:** PostgreSQL seeded with demo users; server and client running (playwright.config.ts).
- **Steps:**

1. Requester logs in via /login (POST /api/auth/login), lands on /requests/my, navigates to /requests/new, and submits an EQUIPMENT request (1800 THB).
2. Browser redirects to /requests/:id, displays PENDING status and SUBMITTED history; reloading the page confirms PostgreSQL persistence (E2E-01).
3. Requester logs out; T Reviewer logs in, lands on /reviewer/pending, opens the assigned request, selects Approve with comment "Approved for project use", and submits (POST /api/requests/:id/review).
4. Navigate back to /reviewer/pending (E2E-02).

- **Expected & Actual Result:** E2E-01 (2.7s) and E2E-02 (1.6s) passed. Status updated to APPROVED, APPROVED by T Reviewer history appended in one transaction, and the request left the pending queue.
- **Evidence Screenshots:**
- !Login Screen (evidence/01-login.png)

- !Requester Detail Pending (evidence/03-requester-detail-pending.png)
- !Reviewer Pending Queue (evidence/04-reviewer-pending-queue.png)
- !Reviewer Approved Detail & History (evidence/05-reviewer-approved-history.png)

9.4.2 Scenario 2: Business-Rule & Boundary Rejection (BR-02, BR-04, BR-07)

- **Feature / AC IDs:** F-02, F-04 (AC-02, AC-08, AC-12)
- **Roles:** REQUESTER, REVIEWER
- **Preconditions:** Authenticated session cookie; existing APPROVED request for double-decision test.
- **Steps:**

1. POST /api/requests with negative/zero amount, >2 decimal places, 2-char title, unknown category, or extra unknown keys (TEST-02).
2. POST /api/requests/:id/review with invalid decision: "ESCALATE", lowercase "approve", REJECT with <3 char comment, or >500 char comment (TEST-12).
3. POST /api/requests/:id/review on an already APPROVED request (TEST-09).

- **Expected & Actual Result:** All invalid payloads returned 400 VALIDATION_ERROR and double-decision returned 400 INVALID_STATE_TRANSITION with zero records created or modified in PostgreSQL (TEST-02, TEST-09, TEST-12 all passed).
- **Evidence Screenshot:**
- !Boundary & Validation Rejection (evidence/02-boundary-validation.png)

9.4.3 Scenario 3: Forbidden & Unauthenticated Access Control (BR-01, BR-03, BR-05)

- **Feature / AC IDs:** F-01, F-03, F-04 (AC-AUTH-03, AC-AUTH-04, AC-04, AC-07, AC-09, AC-10)
- **Roles:** Unauthenticated client, REQUESTER, Manager Hybrid (REVIEWER)
- **Preconditions:** Request owned by Oum Requester assigned to T Reviewer; EQUIPMENT request owned by Manager Hybrid assigned to T Reviewer.
- **Steps & Actual Results:**

1. **Unauthenticated & Post-Logout:** Calling /api/auth/me, POST /api/requests, /api/requests/pending, or /api/requests/:id without cookie or after POST /api/auth/logout → 401 UNAUTHORIZED (TEST-AUTH-03, TEST-04, review.test.ts).
2. **Role Enforcement:** REQUESTER calling POST /api/requests/:id/review or GET /api/requests/pending → 403 FORBIDDEN (TEST-AUTH-04, TEST-10).
3. **Cross-User / Unrelated Reviewer:** Unrelated requester reading GET /api/requests/:id (TEST-04) or Manager Hybrid reviewing a request assigned to T Reviewer (TEST-11) → 403 ACCESS_DENIED.
4. **Self-Approval Guard (BR-01):** Manager Hybrid attempting to approve their own EQUIPMENT request (even though assigned to T Reviewer) → 403 SELF_APPROVAL_NOT_ALLOWED before assignment check (TEST-08, E2E-03 passed).

- **Evidence Screenshot:**
- !Self-Approval Blocked BR-01 (evidence/06-self-approval-blocked.png)

9.5 Actual Test Excerpts (Executed on 2bcbf4a)

9.5.1 Excerpt 1: Domain Rule BR-01 — No Self-Approval (server/tests/review.test.ts:226-256)

Result: **Passed** (395ms in vitest run on 2bcbf4a)

```
it('TEST-08: blocks self-approval before the assignment check (BR-01)', async () => {
  const cookieM = await loginCookie(REVIEWER_M_EMAIL);

  // manager.hybrid owns an EQUIPMENT request, which is assigned to t.reviewer.
  const ownRequest = await createRequest(cookieM, {
    title: 'My Own Equipment Request',
    category: 'EQUIPMENT',
  });

  const dbBefore = await prisma.approvalRequest.findUnique({
    where: { id: ownRequest.id },
    include: { histories: true },
  });
  expect(dbBefore?.assignedReviewerId).toBe(reviewerTId);
  expect(dbBefore?.histories).toHaveLength(1);

  const res = await request(app)
    .post(`/api/requests/${ownRequest.id}/review`)
    .set('Cookie', cookieM)
    .send({ decision: 'APPROVE', comment: 'Self approval attempt' });

  expect(res.status).toBe(403);
  expect(res.body.error.code).toBe('SELF_APPROVAL_NOT_ALLOWED');

  const dbAfter = await prisma.approvalRequest.findUnique({
    where: { id: ownRequest.id },
    include: { histories: true },
  });
  expect(dbAfter?.status).toBe(RequestStatus.PENDING);
  expect(dbAfter?.histories).toHaveLength(1);
});
```

9.5.2 Excerpt 2: Access Restriction BR-03 / BR-05 — Unrelated Reviewer Blocked (server/tests/review.test.ts:311-333)

Result: **Passed** (749ms in `vitest run` on 2bcbf4a)

```
it('TEST-11: rejects an unrelated reviewer with ACCESS_DENIED', async () => {
  const cookieRequester = await loginCookie(REQUESTER_EMAIL);
  const cookieM = await loginCookie(REVIEWER_M_EMAIL);
  const created = await createRequest(cookieRequester, {
    title: 'Not Assigned To M',
    category: 'EQUIPMENT',
  });

  const res = await request(app)
    .post(`/api/requests/${created.id}/review`)
    .set('Cookie', cookieM)
    .send({ decision: 'APPROVE', comment: 'Unrelated reviewer attempt' });

  expect(res.status).toBe(403);
  expect(res.body.error.code).toBe('ACCESS_DENIED');

  const dbRequest = await prisma.approvalRequest.findUnique({
    where: { id: created.id },
    include: { histories: true },
  });
  expect(dbRequest?.status).toBe(RequestStatus.PENDING);
  expect(dbRequest?.histories).toHaveLength(1);
});
```

9.6 TDD Red → Green Evidence

Slice / Issue	Contract Commit	Red Test Commit	Green Implementation Commit	Merge PR
F-01 Auth (#3)	`fc8f548` (2026-11-10) `Contract E0`	`66f13e0` (2026-11-10) `TEST-AUTH-01..05`	`d458119` (2026-11-10) + `0faa578` (2026-11-11)	PR #12 (`ad10d0b`)
F-02/F-03 Requester (#4)	`bb4251b` (2026-11-11) `Contract E1`	`d80cc44` (2026-11-11) `TEST-01. .04`	`331294f` + `37a343b` (2026-11-11)	PR #15 (`fd0c875`)
F-04/F-05 Reviewer (#5)	`4d3731a` (2026-11-12) `Contract E2`	`2c74f2c` (2026-11-12) `TEST-05. .13`	`4f5b2a9` (2026-11-12)	PR #16 (`2bcbf4a`)

9.7 Governance, Contribution & Honest Disclosure

9.7.1 Issue → Branch → PR → Reviewer → Merge Commit

Every change reached `sprint-1-staging` through a reviewed pull request with no author self-approval:

Issue	Scope	Branch	PR	Author	Reviewe	Merge commit
#8	Monorepo scaffold, tooling, base branches	`chore/8-repo-scaffold`	#9	Oum	T	`caa56c7`
#1	Part A/B SRS, feature inventory, traceability	`feature/1-baseline-docs`	#11	Oum	T	`ffcb38b`
#2	Part C/D system SDS and three-sprint plan	`feature/2-system-sds-sprint-plan`	#10	T	Oum	`cf5d4da`
#3	F-01 auth foundation (schema, seed, login/logout/me)	`feature/3-auth-foundation`	#12	T	Oum	`ad10d0b`
#13	Lab 5 Docker image, CI quality gate, Cloud Run pipeline	`feature/13-lab5-docker-cicd`	#14	T	Oum	`001cb0c`
#4	F-02/F-03 requester slice	`feature/4-requester-slice`	#15	Oum	T	`fd0c875`
#5	F-04/F-05 reviewer slice	`feature/5-reviewer-slice`	#16	T	Oum	`2bcbf4a`
#6	Part F Sprint 1 verification and evidence package	`test/6-sprint1-verification`	#17	Oum	T	merge of PR #17

- **Peer Review Rule:** Zero self-approvals; every PR was reviewed and approved by the non-author teammate (`docs/reviewer.md`).
- **Release:** Issue #7 opens the release PR `sprint-1-staging -> main` and tags `v0.1.0-sprint1` after PR #17 merges.

9.7.2 Contribution Table

Member	Student ID	Primary Ownership	Peer Review Ownership
Oum (@ssiriwan)	67070501075	Repo scaffold (#8), Part A/B SRS & Traceability (#1), F-02/F-03 Requester Slice (#4), Part F Verification (#6)	Reviewed PR #10, PR #12, PR #14, PR #16
T (@thhanabun)	67070501021	Part C/D SDS & 3-Sprint Plan (#2), F-01 Auth Foundation (#3), Lab 5 Docker/CI/CD (#13), F-04/F-05 Reviewer Slice (#5)	Reviewed PR #9, PR #11, PR #15

9.7.3 AI-Use Declaration

AI coding assistants were used to accelerate boilerplate scaffolding, test case drafting, and documentation formatting. All architecture decisions, business-rule enforcement orders, database schemas, and test verifications were reviewed, executed locally and in CI, and peer-reviewed by both team members.

9.7.4 Known Limitations & Honest Disclosure

1. **Deferred Sprint 2 / Sprint 3 Scope:** FR-07 (Return for Revision), FR-08 (Multi-stage Approval), FR-09 (Attachments), FR-10 (Search/Filter/Reporting), and FR-11 (Admin User/Role Management) are intentionally out of Sprint 1 scope and remain Planned for Sprints 2–3.
2. **NFR-02 Load Benchmark:** Formal 50-concurrent-request p95 latency load testing is deferred to Sprint 3; Sprint 1 verified functional correctness and single-query/transactional indexing.
3. **Playwright E2E in CI:** E2E-01..03 pass locally against PostgreSQL + Vite + Express (3/3 in 11.7s), while .github/workflows/ci-docker.yml currently runs the 34 Vitest unit/integration tests, TypeScript builds, and Docker container /api/health probe.
4. **Minor Sprint 1 Polish Backlog:** Contract E1 §2 mentions a createdAt column in MyRequestsPage that is not rendered in the Sprint 1 table; requestNo uses a UTC date prefix (YYYYMMDD); category-to-reviewer mapping (D-03) is static until Sprint 3 Admin Management (FR-11).